



2025-여름학기 비교과 프로그램 게임온라인공채대비 코딩캠프

강의시간 : 42시간(7일x6시간)

8월19일~8월22일(4일), 8월26일~8월28일(3일) (10:00~16:00)

Zoom 실시간 강의

<https://us02web.zoom.us/j/5233405142?>

pwd=UXVRVFFvRWNQaEVNTDN5SFJvZHhTQT09

담당교수 : 김영식

코딩으로 취업하자!!

- 강좌구분
 - 특성화 프로그램(비교과)특강
- 목표
 - 다양한 난이도와 중급 규모 이상의 실전 문제 코딩 프로그래밍을 통하여 코딩 실력을 향상시킨다.
 - BFS/DFS, 다이나믹프로그래밍, 구현, 탐욕법, 브루트포스, 최단거리 (Dijkstra), 이분탐색
- 기대효과
 - 코딩 역량을 향상시킬 수 있으며 이를 통하여 게임 및 IT 관련 기업의 코딩 공채 테스트에 대비할 수 있다
- 교재
 - 백준 온라인저지 <https://www.acmicpc.net/>

특강 계획(Python, C++)

일차	내용
1일 08/19 화 (10:00~16:00)	구현, 브루트포스, 조합, 정렬, 해시, 자료구조 블랙잭, 덩치, 창고다각형, 빗물, 영단어암기는 괴로워, 에디터
2일 08/20 수 (10:00~16:00)	동적 계획법 (Dynamic Programming) 설탕배달, 123더하기4, 1로만들기, 진우의달여행, 계단오르기
3일 08/21 목 (10:00~16:00)	깊이우선탐색 (DFS) / 너비우선탐색 (BFS) DFS와BFS, 쉬운최단거리, 숨바꼭질, 위성이미지, 블록이동하기
4일 08/22 금 (10:00~16:00)	DFS/BFS, 우선순위큐(힙) 탈옥, 더 맵게, 보석도둑, 디스크 컨트롤러
5일 08/26 화 (10:00~16:00)	다익스트라(Dijkstra), 플로이드워셜 지름길, 숨바꼭질3, K번째최단경로찾기, 장애물, 합승택시요금
6일 08/27 수 (10:00~16:00)	투포인터, 슬라이딩윈도우, 누적합 두수의합, 회전초밥, 겹치는건싫어, 블로그, 목재총량
7일 08/28 목 (10:00~16:00)	이분 탐색 (Binary Search) 나무자르기, 셔틀버스대기시간, 공유기설치

1. 블랙잭 (백준2798, 조합, 브루트포스)

<https://www.acmicpc.net/problem/2798>

- 제한된 시간 안에 **N장의 카드 중에서 3장의 카드를 골라야 한다**. 블랙잭 변형 게임이기 때문에, 플레이어가 고른 카드의 합은 M을 넘지 않으면서 M과 최대한 가깝게 만들어야 한다.
- N장의 카드에 써져 있는 숫자가 주어졌을 때, M을 넘지 않으면서 M에 최대한 가까운 카드 3장의 합을 구해 출력하시오.**
- 첫째 줄에 카드의 개수 $N(3 \leq N \leq 100)$ 과 $M(10 \leq M \leq 300,000)$ 이 주어진다.** 둘째 줄에는 카드에 쓰여 있는 수가 주어지며, 이 값은 100,000을 넘지 않는 양의 정수이다.

예제 입력 1 복사

```
5 21
5 6 7 8 9
```

예제 출력 1 복사

```
21
```

순열(permutations)과 조합(combinations)

- 파이썬 **itertools**

```
from itertools import permutations
from itertools import combinations

items= [1,2,3,4]
print(list(combinations(items,2))): # items 에서 2개 선택 조합
# [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
print(list(permutations(items,2))): # items 에서 2개 선택 순열
# [(1, 2), (1, 3), (1, 4), (2, 1), (2, 3), (2, 4), (3, 1), (3, 2), (3, 4)
, (4, 1), (4, 2), (4, 3)]
```

순열(permutations)과 조합(combinations)

- 조합(combinations) 알고리즘
 - combination([1,2,3,4],2) = ([1] + combination([2,3,4],1)) and ([2] + combination([3,4],1)) and ([3] + combination([4],1))

```
def comb(lst,n):  
    ret = []  
    if n > len(lst): # 예외 처리  
  
        return ret  
  
    if n == 1:  
        for i in lst:  
            ret.append([i])  
    elif n>1:  
        for i in range(len(lst)-n+1): #i=0,1,2,...,L-n (L-n+1번 반복)  
            for temp in comb(lst[i+1:],n-1): #lst[L-n+1:] : n-1개 원소  
                ret.append([lst[i]]+temp)
```

return ret

순열(permutations)과 조합(combinations)

- 순열(permutations) 알고리즘
 - $\text{permutation}([1,2,3,4],2) = ([1] + \text{permutation}([2,3,4],1)) \text{ and } ([2] + \text{permutation}([1,3,4],1)) \text{ and } ([3] + \text{permutation}([1,2,4],1)) \text{ and } ([4] + \text{permutation}([1,2,3],1))$

```
def perm(lst,n):  
    ret = []  
    if n > len(lst): # 예외 처리  
        return ret  
    if n==1:  
        for i in lst:  
            ret.append([i])  
    elif n>1:  
        for i in range(len(lst)): #i=0,1,2,...,L-1  
            temp = lst[:] # 복사  
            temp.remove(lst[i]) #i번째 원소 제거  
            for p in perm(temp,n-1):  
                ret.append([lst[i]]+p)
```

2. 덩치 (백준7568, 브루트포스)

<https://www.acmicpc.net/problem/7568>

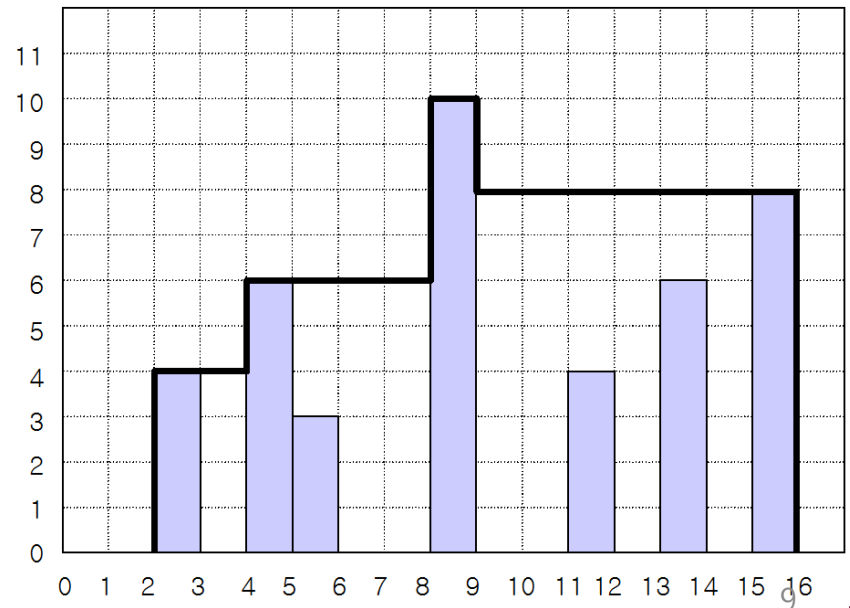
- 어떤 사람의 몸무게가 x kg이고 키가 y cm라면 이 사람의 덩치는 (x, y) 로 표시된다. 두 사람 A와 B의 덩치가 각각 (x, y) , (p, q) 라고 할 때 $x > p$ 그리고 $y > q$ 이라면 우리는 A의 덩치가 B의 덩치보다 "더 크다"고 말한다.
- N명의 집단에서 각 사람의 덩치 등수는 자신보다 더 "큰 덩치"의 사람의 수로 정해진다. 만일 자신보다 더 큰 덩치의 사람이 k명이라면 그 사람의 덩치 등수는 $k+1$ 이 된다. 이렇게 등수를 결정하면 같은 덩치 등수를 가진 사람은 여러 명도 가능하다.

이름	(몸무게, 키)	덩치 등수
A	(55, 185)	2
B	(58, 183)	2
C	(88, 186)	1
D	(60, 175)	2
E	(46, 155)	5

3. 창고다각형 (백준2304, 정렬, 구현)

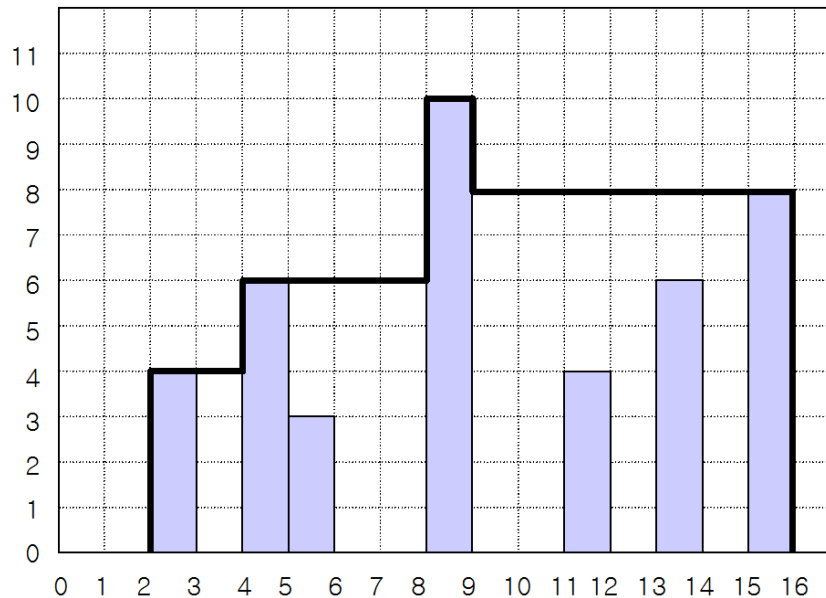
<https://www.acmicpc.net/problem/2304>

- N 개의 막대 기둥이 일렬로 세워져 있다. 기둥들의 폭은 모두 1 m이며 높이는 다를 수 있다. 이 기둥들을 이용하여 양철로 된 창고를 제작하려고 한다. 창고에는 모든 기둥이 들어간다. 이 창고의 지붕을 다음과 같이 만든다.
- 지붕은 수평 부분과 수직 부분으로 구성되며, 모두 연결되어야 한다.
- 지붕의 수평 부분은 반드시 어떤 기둥의 윗면과 닿아야 한다.
- 지붕의 수직 부분은 반드시 어떤 기둥의 옆면과 닿아야 한다.
- 지붕의 가장자리는 땅에 닿아야 한다.
- 오목하게 들어간 부분이 없어야 한다.



3. 창고다각형 (백준2304, 정렬, 구현) 문제해결 아이디어

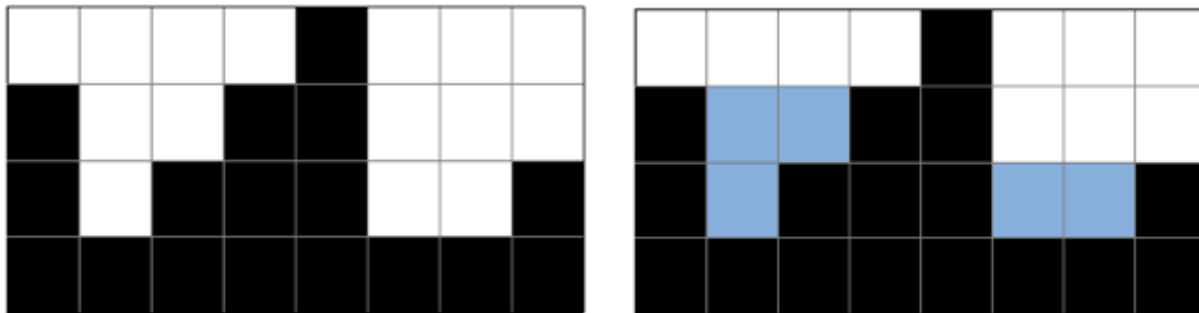
- 위치 기준 정렬
- 최고 높이 위치 찾기
- 제일 왼쪽에서 오른쪽으로 최고 높이 위치까지 오름차순 지붕 설치
- 제일 오른쪽에서 왼쪽으로 최고 높이 위치까지 오름차순 지붕 설치



4. 빗물 (백준14719, 구현)

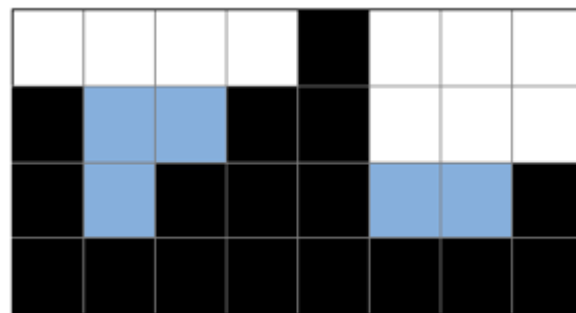
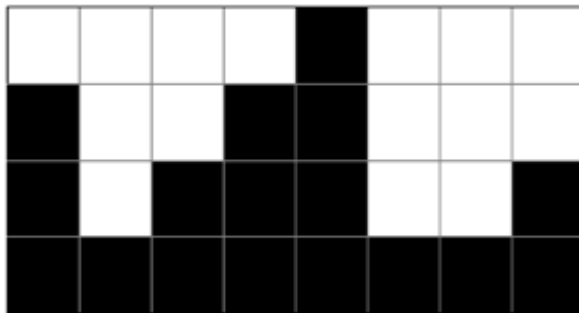
<https://www.acmicpc.net/problem/14719>

- 2차원 세계에 블록이 쌓여있다. 비가 오면 블록 사이에 빗물이 고인다.
- 첫 번째 줄에는 세로 길이 h 와 가로 길이 w 가 주어진다. ($1 \leq h, w \leq 500$)
- 두 번째 줄에는 블록이 쌓인 높이를 의미하는 0 이상 h 이하의 정수가 맨 왼쪽 위치부터 차례대로 w 개 주어진다.
- 따라서 블록 내부의 빈 공간이 생길 수 없다.
- 한 칸의 용량은 1 이다. 고이는 빗물의 총량을 출력하여라.
- 빗물이 전혀 고이지 않을 경우 0 을 출력하여라.



4. 빗물 (백준14719, 구현) 문제해결 아이디어

- 창고다각형(백준2304) 문제와 유사



5. 영단어 암기는 괴로워 (백준20920, 정렬, 해시)

<https://www.acmicpc.net/problem/20920>

- 단어장의 단어 순서는 다음과 같은 우선순위를 차례로 적용하여 만들어진다.
- 자주 나오는 단어일수록 앞에 배치한다.
- 해당 단어의 길이가 길수록 앞에 배치한다.
- 알파벳 사전 순으로 앞에 있는 단어일수록 앞에 배치한다
-
- M보다 짧은 길이의 단어의 경우 읽는 것만으로도 외울 수 있기 때문에 길이가 M이상인 단어들만 외운다고 한다. 영단어 암기를 효율적으로 할 수 있도록 단어장을 만들어 주자.

예제 입력 1 복사

```
7 4
apple
ant
sand
apple
append
sand
sand
```

예제 출력 1 복사

```
sand
apple
append
```

6. 에디터 (백준1406, 자료구조)

<https://www.acmicpc.net/problem/1406>

- 한 줄로 된 간단한 에디터
- 이 편집기에는 '커서'라는 것이 있는데, 커서는 문장의 맨 앞(첫 번째 문자의 왼쪽), 문장의 맨 뒤(마지막 문자의 오른쪽), 또는 문장 중간 임의의 곳(모든 연속된 두 문자 사이)에 위치할 수 있다.
- 길이가 L 인 문자열, 커서가 위치할 수 있는 곳은 $L+1$ 가지 경우.
- 모든 명령어를 수행하고 난 후 편집기에 입력되어 있는 문자열을 출력한다.

L	커서를 왼쪽으로 한 칸 옮김 (커서가 문장의 맨 앞이면 무시됨)
D	커서를 오른쪽으로 한 칸 옮김 (커서가 문장의 맨 뒤이면 무시됨)
B	커서 왼쪽에 있는 문자를 삭제함 (커서가 문장의 맨 앞이면 무시됨) 삭제로 인해 커서는 한 칸 왼쪽으로 이동한 것처럼 나타나지만, 실제로 커서의 오른쪽에 있던 문자는 그대로임
P \$	\$라는 문자를 커서 왼쪽에 추가함

6. 에디터 (백준1406, 자료구조)

문제해결 아이디어

- 리스트 1개에 커서 관리 문제점 : 중간 문자 삽입/삭제가 $O(N)$ 으로 시간 초과
- 파이썬 deque 사용
- 리스트 2개 관리, 커서는 2개 리스트 사이에 존재 : 문자 삽입/삭제 $O(1)$
- $[a, b, c] * [d]$

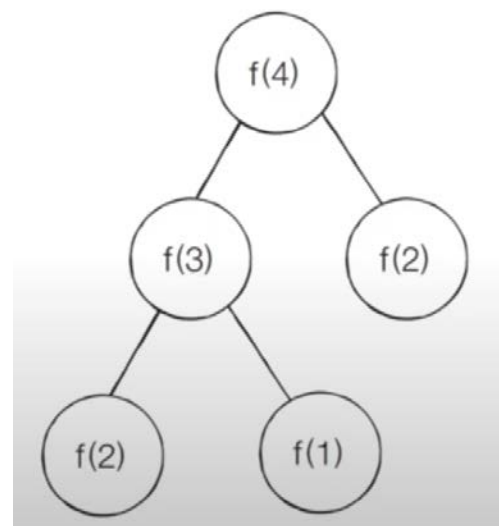
동적 계획법(Dynamic Programming)

- 문제를 여러 개의 **하위 문제(subproblem)**로 나눈다.
- 하위문제 결과들을 결합하여 최종적인 목적에 도달한다.
- **Memoization** : 각 하위 문제의 해결을 계산한 뒤, 그 해결책을 저장하여 후에 같은 하위 문제가 나왔을 경우 그것을 간단하게 해결할 수 있다.

피보나치 수열 : 다이나믹 프로그래밍

- 다이나믹 프로그래밍의 사용 조건
 1. 최적 부분 구조 (Optimal Substructure)
 - 큰 문제를 작은 문제로 나눌 수 있습니다.
 2. 중복되는 부분 문제 (Overlapping Subproblem)
 - 동일한 작은 문제를 반복적으로 해결해야 합니다.
- 피보나치 수열은 다이나믹 프로그래밍 사용 조건을 만족
 - 점화식 (인접한 항들 사이의 관계식)

$$F_n = F_{n-1} + F_{n-2}, \quad F_1 = 1, F_2 = 1$$



탑다운 VS 바텀업

- 탑다운(메모이제이션) 방식은 하향식
- 바텀업(DP 테이블) 방식은 상향식
- 다이나믹 프로그래밍의 전형적인 형태는 바텀업 방식

7. 설탕배달 (백준2839, DP)

<https://www.acmicpc.net/problem/2839>

- 설탕을 정확하게 **N킬로그램**을 배달해야 한다. 설탕공장에서 만드는 설탕은 봉지에 담겨져 있다. 봉지는 **3킬로그램 봉지**와 **5킬로그램 봉지**가 있다.
- 상근이는 귀찮기 때문에, **최대한 적은 봉지**를 들고 가려고 한다. 예를 들어, 18킬로그램 설탕을 배달해야 할 때, 3킬로그램 봉지 6개를 가져가도 되지만, 5킬로그램 3개와 3킬로그램 1개를 배달하면, 더 적은 개수의 봉지를 배달할 수 있다.
- 상근이가 설탕을 정확하게 **N킬로그램** 배달해야 할 때, 봉지 몇 개를 가져가면 되는지 그 수를 구하는 프로그램을 작성하시오.

예제 입력 1 복사

18

예제 출력 1 복사

4

7. 설탕배달 (백준2839, DP)

문제 해결 아이디어

- a_i = 설탕 i 를 만들기 위한 최소한의 봉지 개수
- k = 각 봉지의 단위
- **점화식:** 각 봉지 단위인 k 를 하나씩 확인하여
 - a_{i-k} 를 만드는 방법이 존재하는 경우, $a_i = \min(a_i, a_{i-k} + 1)$
 - a_{i-k} 를 만드는 방법이 존재하지 않는 경우, $a_i = \text{INF}$

7. 설탕배달 (백준2839, DP)

문제 해결 아이디어

- 봉지개수 3개, 설탕 7킬로그램 배달
 - 각 봉지의 단위가 2, 3, 5킬로인 경우
- Step 0 (초기화)
 - 먼저 각 인덱스에 해당하는 값으로 INF(무한)의 값을 설정
 - INF는 특정 단위를 만들 수 있는 봉지 구성이 가능하지 않다는 의미
 - 본 문제에서는 10,001을 사용할 수 있다.

인덱스	0	1	2	3	4	5	6	7
값	0	10,001	10,001	10,001	10,001	10,001	10,001	10,001

7. 설탕배달 (백준2839, DP)

문제 해결 아이디어

- 봉지개수 3개, 설탕 7킬로그램 배달
 - 각 봉지의 단위가 2, 3, 5킬로인 경우
- Step 1
 - 첫 번째 봉지 단위인 2를 확인
 - 점화식에 따라서 다음과 같이 리스트가 갱신

인덱스	0	1	2	3	4	5	6	7
값	0	10,001	1	10,001	2	10,001	3	10,001

4원을 만들기 위한 개수는 2개: (2원 + 2원)

7원을 만드는 방법이 없음

7. 설탕배달 (백준2839, DP)

문제 해결 아이디어

- 봉지개수 3개, 설탕 7킬로그램 배달
 - 각 봉지의 단위가 2, 3, 5킬로인 경우
- Step 2
 - 두 번째 봉지 단위인 3을 확인
 - 점화식에 따라서 다음과 같이 리스트가 갱신

인덱스	0	1	2	3	4	5	6	7
값	0	10,001	1	1	2	2	2	3

7원을 만들기 위한 개수는
3개: (2원 + 2원 + 3원)

7. 설탕배달 (백준2839, DP)

문제 해결 아이디어

- 봉지개수 3개, 설탕 7킬로그램 배달
 - 각 봉지의 단위가 2, 3, 5킬로인 경우
- Step 3
 - 세 번째 봉지 단위인 5를 확인
 - 점화식에 따라서 다음과 같이 리스트가 갱신

인덱스	0	1	2	3	4	5	6	7
값	0	10,001	1	1	2	1	2	2

7원을 만들기 위한 개수는
2개: (2원 + 5원)

8. 1,2,3더하기 4 (백준15989, DP)

<https://www.acmicpc.net/problem/15989>

- 정수 4를 1, 2, 3의 합으로 나타내는 방법은 총 4가지가 있다. 합을 나타낼 때는 수를 1개 이상 사용해야 한다. 합을 이루고 있는 수의 순서만 다른 것은 같은 것으로 친다.
- 1+1+1+1
- 2+1+1 (1+1+2, 1+2+1)
- 2+2
- 1+3 (3+1)
- 정수 n 이 주어졌을 때, n 을 1, 2, 3의 합으로 나타내는 방법의 수를 구하는 프로그램 작성하시오.

8. 1,2,3더하기 4 (백준15989, DP)

문제 해결 아이디어

- $a_i = i$ 를 1,2,3 합으로 나타내는 방법의 수
- $a_i = 1$ (초기화, $1+1+1+\dots,+1 = i$)
- 점화식 ($k=2,3$)
- $a_i = a_{i-k} + a_i$ ($i-k$ 를 만드는 방법에 k 를 1개 더하는 방법을 추가)

9. 1로 만들기 (백준1463, DP)

<https://www.acmicpc.net/problem/1463>

- 정수 x 에 사용할 수 있는 연산은 다음과 같이 세 가지 이다.
 - x 가 3으로 나누어 떨어지면, 3으로 나눈다.
 - x 가 2로 나누어 떨어지면, 2로 나눈다.
 - 1을 뺀다.
-
- 정수 N 이 주어졌을 때, 위와 같은 연산 세 개를 적절히 사용해서 1을 만들려고 한다. 연산을 사용하는 횟수의 최솟값을 출력하시오.

예제 입력 2 복사

10

예제 출력 2 복사

3

9. 1로 만들기 (백준1463, DP)

문제 해결 아이디어

- a_i = i 를 1로 만들기 위한 최소 연산 횟수
- $a_i = 0$ 으로 초기화
- 점화식은 다음과 같다.

$$a_i = \min(a_{i-1}, a_{i/2}, a_{i/3}, a_{i/5}) + 1$$

- 단 1을 빼는 연산을 제외하고는 해당 수를 나누어 떨어질 때에 한해 점화식을 적용할 수 있다.

10. 진우의 달 여행 (백준17484, DP)

<https://www.acmicpc.net/problem/17484>

- 지구 -> 달로 가는 경우 우주선이 움직일 수 있는 방향은 아래와 같다.



- 우주선은 전에 움직인 방향으로 움직일 수 없다. 즉, 같은 방향으로 두번 연속으로 움직일 수 없다.
- 진우의 목표는 연료를 최대한 아끼며 지구의 어느 위치에서든 출발하여 달의 어느 위치든 착륙하는 것이다.
- 최대한 돈을 아끼고 살아서 달에 도착하고 싶은 진우를 위해 달에 도달하기 위해 필요한 연료의 최소값을 계산해 주자.

지구			
5	8	5	1
3	5	8	4
9	77	65	5
2	1	5	2
5	98	1	5
4	95	67	58
달			

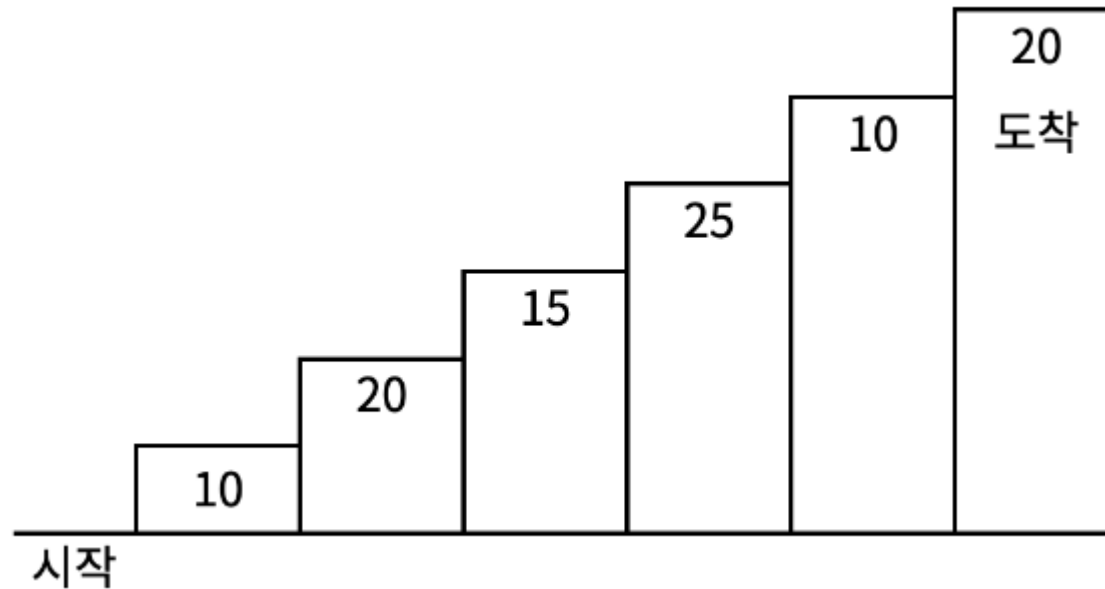
10. 진우의 달 여행 (백준17484, DP)

문제 해결 아이디어

- $array[i][j]$ = i행 j열 연료
- $dp[i][j]$ = (i행 j열까지의 최적의 해 (최솟값), 움직인 방향 (L,U,R))
- 점화식
$$dp[i][j] = array[i][j] + \min(dp[i-1][j-1], dp[i-1][j], dp[i-1][j+1])$$
- 이때 테이블에 접근할 때마다 리스트의 범위를 벗어나지 않는지 체크
- 같은 경로를 2번 연속 갈 수 없으므로 dp테이블의 값을 최적해와 움직인 방향 (L, U, R)을 저장해서 값을 갱신할 때 고려한다.

11. 계단오르기 (백준2579, DP)

<https://www.acmicpc.net/problem/2579>



계단 오르기 게임은 계단 아래 시작점부터 계단 꼭대기에 위치한 도착점까지 가는 게임이다. 그림과 같이 각각의 계단에는 일정한 점수가 쓰여 있는데 계단을 밟으면 그 계단에 쓰여 있는 점수를 얻게 된다.

11. 계단오르기 (백준2579, DP)

문제 해결 아이디어

점화식

그렇다면, 마지막 계단을 밟았을 경우, 이전의 경우를 2가지로 생각할 수 있다.

1. 마지막 계단 전의 계단을 밟은 경우.
2. 마지막 계단 전의 계단을 밟지 않은 경우.

1번의 경우에는 마지막 계단 전의 계단을 밟았음으로, 마지막 계단 전전의 계단은 밟지 못한다.

2번의 경우에는 마지막 계단 전의 계단을 밟지 않았음으로, 마지막 계단 전전의 계단을 밟고 왔다.

dp 배열은 합을 구하는 배열이고, array 배열은 계단의 값이다.

$$1. dp[n] = dp[n-3] + array[n-1] + array[n]$$

$$2. dp[n] = dp[n-2] + array[n]$$

문제는 답은 최대값을 찾는 것이기 때문에 위 점화식 2개를 비교하면서 큰 값을 넣으면 된다.

$$dp[n] = \max(dp[n-3] + array[n-1] + array[n], dp[n-2] + array[n])$$

Tree Traversal

preorder(node) “깊이우선탐색(DFS)”

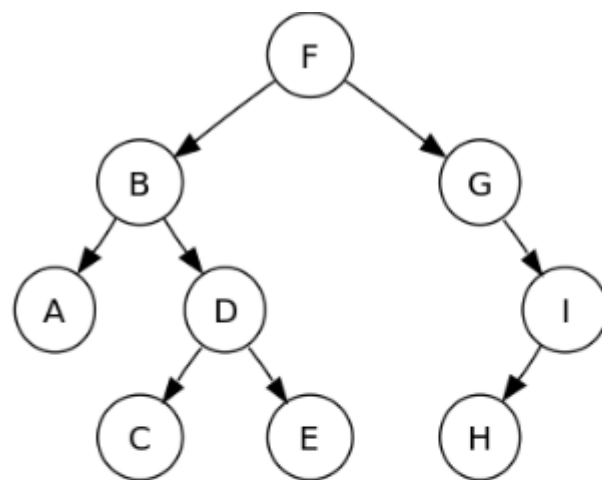
```
print node.value
if node.left ≠ null then preorder(node.left)
if node.right ≠ null then preorder(node.right)
```

inorder(node)

```
if node.left ≠ null then inorder(node.left)
print node.value
if node.right ≠ null then inorder(node.right)
```

postorder(node) “리프노드에서 위로 탐색”

```
if node.left ≠ null then postorder(node.left)
if node.right ≠ null then postorder(node.right)
print node.value
```

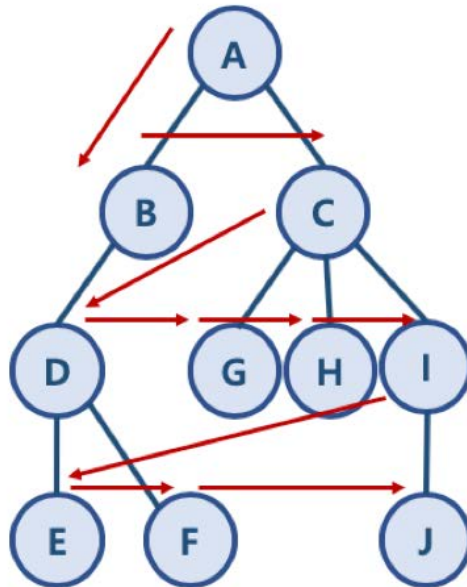


이진 탐색 트리에서

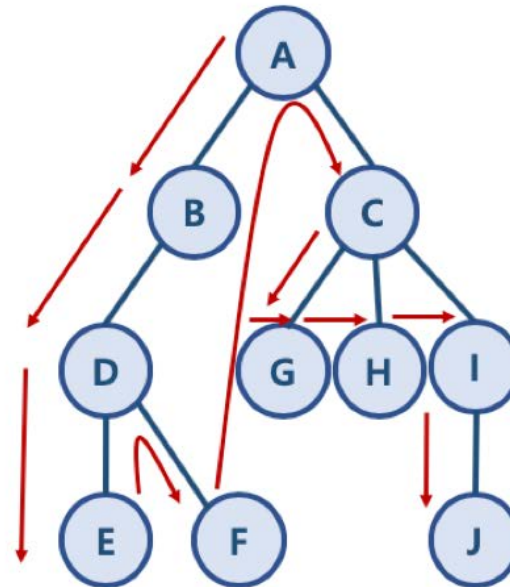
- 전위 순회: F, B, A, D, C, E, G, I, H (root, left, right)
- 중위 순회: A, B, C, D, E, F, G, H, I (left, root, right)
- 후위 순회: A, C, E, D, B, H, I, G, F (left, right, root)
- 레벨 순서 순회: F, B, G, A, D, I, C, E, H

깊이우선탐색(DFS), 너비우선탐색(BFS)

- Depth First Search(전위순회 preorder traversal): **스택, 재귀**
- Breadth First Search: **큐**
- 완전탐색(그래프의 모든 정점 방문) : **DFS / BFS**
- 경로의 특징 저장 (경로에 같은 숫자가 있으면 안됨): **DFS**
- 미로 찾기 (최단거리) : **BFS**



게임온라인공채대비 코딩캠프 **Breadth First Search**



Depth First Search

12. DFS/BFS DFS와 BFS (백준1260, DFS/BFS)

<https://www.acmicpc.net/problem/1260>

- 그래프를 DFS로 탐색한 결과와 BFS로 탐색한 결과를 출력하는 프로그램을 작성하시오.
- 단, 방문할 수 있는 정점이 여러 개인 경우에는 정점 번호가 작은 것을 먼저 방문하고, 더 이상 방문할 수 있는 점이 없는 경우 종료한다.
- 정점 번호는 1번부터 N번까지이다.

```
# 1          4 5 1
# |_____  
# |  |  |  
# 2  3__4  
# |_____|  
          1 2  
          1 3  
          1 4  
          2 4  
          3 4
```

정점의 개수 $N(1 \leq N \leq 1,000)$, 간선의 개수 $M(1 \leq M \leq 10,000)$, 탐색을 시작할 정점의 번호 V 가 주어진다.

13. 쉬운최단거리 (백준14940, BFS)

<https://www.acmicpc.net/problem/14940>

- 지도가 주어지면 **모든 지점에 대해서 목표지점까지의 거리를 구하여라.**
- 문제를 쉽게 만들기 위해 오직 가로와 세로로만 움직일 수 있다고 하자.
- 지도의 크기 n 과 m 이 주어진다. n 은 세로의 크기, m 은 가로의 크기다. ($2 \leq n \leq 1000, 2 \leq m \leq 1000$)
- 다음 n 개의 줄에 m 개의 숫자가 주어진다. **0은 갈 수 없는 땅이고 1은 갈 수 있는 땅, 2는 목표지점이다. 입력에서 2는 단 한개이다.**
- 각 지점에서 목표지점까지의 거리를 출력한다. **원래 갈 수 없는 땅인 위치는 0을 출력하고, 원래 갈 수 있는 땅인 부분 중에서 도달할 수 없는 위치는 -1을 출력한다.**

13. 쉬운최단거리 (백준14940, BFS)

<https://www.acmicpc.net/problem/14940>

예제 입력 1 복사

```
15 15
2 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 0 0 0 0 1
1 1 1 1 1 1 1 1 1 1 0 1 1 1 1
1 1 1 1 1 1 1 1 1 1 0 1 0 0 0
1 1 1 1 1 1 1 1 1 1 0 1 1 1 1
```

예제 출력 1 복사

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
11 12 13 14 15 16 17 18 19 20 0 0 0 0 25
12 13 14 15 16 17 18 19 20 21 0 29 28 27 26
13 14 15 16 17 18 19 20 21 22 0 30 0 0 0
14 15 16 17 18 19 20 21 22 23 0 31 32 33 34
```

14. 숨바꼭질 (백준1697, BFS)

<https://www.acmicpc.net/problem/1697>

- 수빈이는 동생과 숨바꼭질을 하고 있다. 수빈이는 현재 점 N ($0 \leq N \leq 100,000$)에 있고, 동생은 점 K ($0 \leq K \leq 100,000$)에 있다.
- 수빈이는 걷거나 순간이동을 할 수 있다. 만약, 수빈이의 위치가 X 일 때 걷는다면 1초 후에 $X-1$ 또는 $X+1$ 로 이동하게 된다. 순간이동을 하는 경우에는 1초 후에 $2 \times X$ 의 위치로 이동하게 된다.
- 수빈이와 동생의 위치가 주어졌을 때, 수빈이가 동생을 찾을 수 있는 가장 빠른 시간이 몇 초 후인지 구하는 프로그램을 작성하시오.
- 첫 번째 줄에 수빈이가 있는 위치 N 과 동생이 있는 위치 K 가 주어진다. N 과 K 는 정수이다.

15. 위성이미지(TUKorea0052, DFS/BFS)

<http://210.93.60.51/problem/0052>

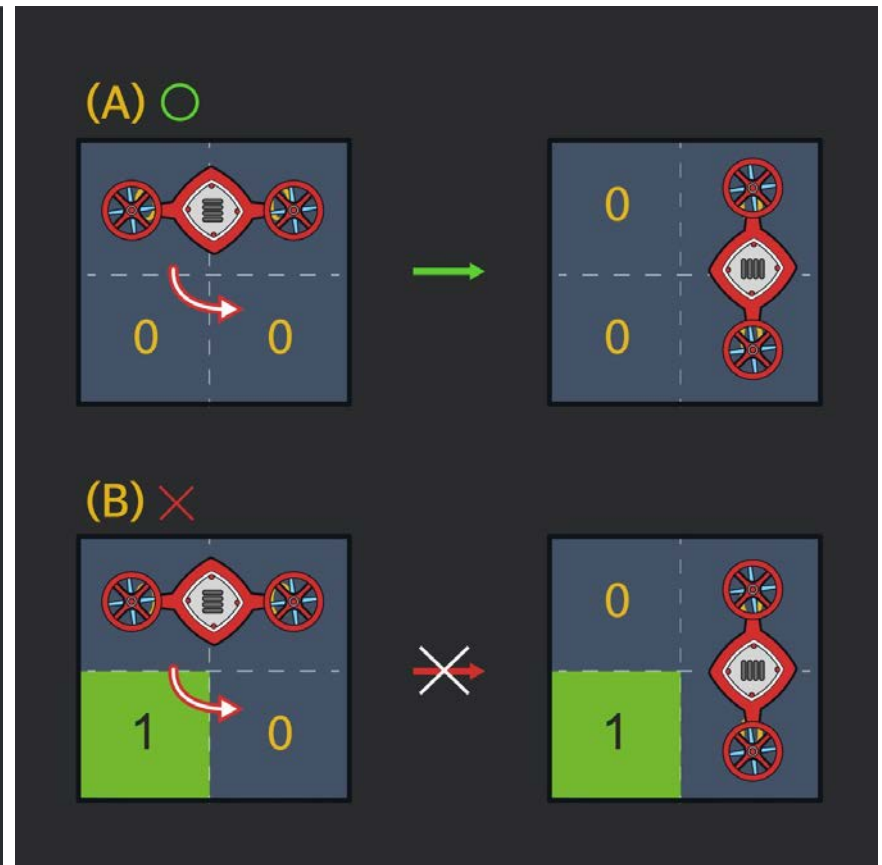
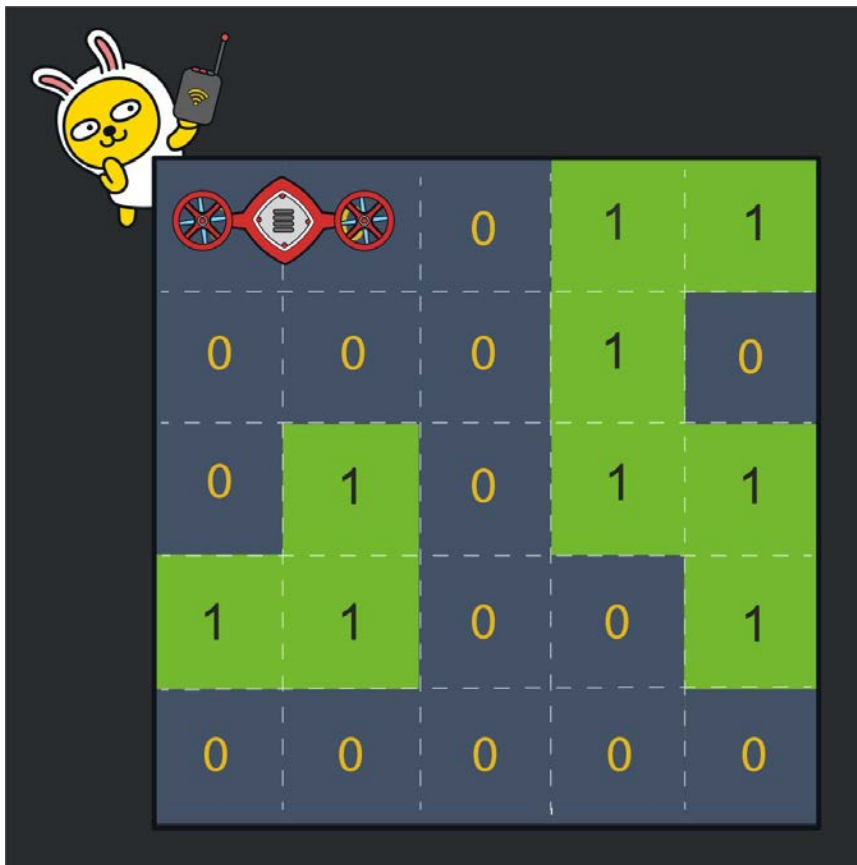
- 폭풍으로 간주할 임계 크기를 4라고 하면, 크기가 4를 넘어가는 구름은 2개가 있으며 아래와 같이 숫자 1과 2의 영역으로 표시할 수 있습니다.
- 완전탐색 : DFS/BFS

```
#####1#####  
#####1#####.#  
###11##.#.#  
##111#####  
#####22222  
#####
```

16. 블록 이동하기 (Programmers, BFS)

<https://programmers.co.kr/learn/courses/30/lessons/60063>

- 2020 카카오공채 1차 7번



16. 블록 이동하기 (Programmers, BFS)

<https://programmers.co.kr/learn/courses/30/lessons/60063>

```
##### 가로 #####
#      c-2  c  c+1 c+2
#  r-1  0  0  0  0
#  r    0  A  B  0
#  r+1  0  0  0  0
#  r+2  0  0  0  0
```

```
# 왼쪽(A)축 시계방향 회전
# AB  -> A0
# 00  B0
```

```
# 왼쪽(A)축 반시계방향 회전
# 00  B0
# AB  -> A0
```

```
# 오른쪽(B)축 시계방향 회전
# 00  0B
# AB  -> 0A
```

```
# 오른쪽(B)축 반시계방향 회전
# AB  -> 0B
# 00  0A
```

16. 블록 이동하기 (Programmers, BFS) 문제해결 아이디어

세로

#		c-2	c	c+1	c+2
#	r-1	0	0	0	0
#	r	0	A	0	0
#	r+1	0	B	0	0
#	r+2	0	0	0	0

위쪽(A)측 시계방향 회전
0A -> BA
0B 00

위쪽(A)측 반시계방향 회전
A0 -> AB
B0 00

아래쪽(B)측 시계방향 회전
A0 -> 00
B0 BA

아래쪽(B)측 반시계방향 회전
0A -> 00
0B AB

17. 탈옥 (백준9376, BFS)

<https://www.acmicpc.net/problem/9376>

- 빈 공간은 '.', 지나갈 수 없는 벽은 '*', 문은 '#', 죄수의 위치는 '\$'이다.
- 두 죄수를 탈옥시키기 위해서 열어야 하는 문의 최소 개수를 구하는 프로그램

문제해결 아이디어

STEP1 Map 바깥을 전부 . 으로 통과할 수 있는 것으로 둘러싼다.

```
# h = 5, w = 9
# .....
# .***#***.
# .*..#.*.
# .****.****.
# .*$#.#.$*.
# .*****.
# .....
```

17. 탈옥 (백준9376, BFS)

문제해결 아이디어

```
# Dis0 : 바깥 (0,0) 에서 모든 포인트의 문 통과 개수를 구하고  
# Dis1 : prisoner1 에서 모든 포인트의 문 통과 개수를 구하고  
# Dis2 : prisoner2 에서 모든 포인트의 문 통과 개수를 구한다.
```

```
# 두 사람은 어느 문 지점까지 각자 이동하고 그 지점부터는 같은 루트로 탈출한다.  
# prisoner1 -> (x,y), prisoner -> (x,y), (x,y)->바깥(0,0)  
# 모든 지점에 대해서 Dis0 + Dis1 + Dis2 가 최저가 되는 것을 찾는다.
```

17. 탈옥 (백준9376, BFS)

문제해결 아이디어

```
# BFS 큐
# 풀이가 진짜 독특하다. 난이도 최강
# 큐를 2개 써서 문을 통과하기 전까지 큐를 탐색하고 문을 만나면 다른 큐에 넣었다가
# 첫번째 큐를 다 탐색하면 두번째 큐 (즉 문 하나 통과한 큐)를 탐색한다.
# (sx,sy) 시작점에서 h*w 맵 D의 모든 포인트 . 까지 문 통과 개수를 계산
```

```
# h = 5, w = 9
# .....
# .*****#*****.
# .*..#.#..*.
# .*****.*****.
# .*$#.#.#$*.
# .*****.
# .....
```

Dis0

```
# [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
# [0, -1, -1, -1, -1, 0, -1, -1, -1, -1, 0]
# [0, -1, 2, 2, 1, 1, 1, 2, 2, -1, 0]
# [0, -1, -1, -1, -1, 1, -1, -1, -1, -1, 0]
# [0, -1, 3, 2, 2, 1, 2, 2, 3, -1, 0]
# [0, -1, -1, -1, -1, -1, -1, -1, -1, -1, 0]
# [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

우선순위 큐 (Priority Queue)

- 우선순위가 가장 높은 데이터를 가장 먼저 삭제하는 자료구조
- 예를 들어 여러 개의 물건 데이터를 자료구조에 넣었다가 가치가 높은 물건 데이터부터 꺼내서 확인해야 하는 경우에 우선순위 큐를 이용할 수 있다.
- Python, C++, Java를 포함한 대부분의 프로그래밍 언어에서 **표준 라이브러리 형태로 지원**

자료구조	추출되는 데이터
스택(Stack)	가장 나중에 삽입된 데이터
큐(Queue)	가장 먼저 삽입된 데이터
우선순위 큐(Priority Queue)	가장 우선순위가 높은 데이터

힙 (Heap)

- 우선순위 큐 (Priority Queue)를 구현하기 위해 사용하는 자료구조 중 하나
- 최소 힙 (Min Heap)과 최대 힙 (Max Heap)이 있다.
- 다익스트라 최단 경로 알고리즘을 포함한 다양한 알고리즘에 사용

우선순위 큐 구현 방식	삽입 시간	삭제 시간
리스트	$O(1)$	$O(N)$
힙(Heap)	$O(\log N)$	$O(\log N)$

힙 라이브러리 사용 예제: 최소 힙

```
import heapq
# 오름차순 힙 정렬(Heap Sort)
def heapsort(iterable):
    h = []
    result = []
    # 모든 원소를 차례대로 힙에 삽입
    for value in iterable:
        heapq.heappush(h, value)
    # 힙에 삽입된 모든 원소를 차례대로 꺼내어 담기
    for i in range(len(h)):
        result.append(heapq.heappop(h))
    return result

result = heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0])
print(result)
```

실행 결과

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```


힙 라이브러리 사용 예제: 최대 힙

```
import heapq

# 내림차순 힙 정렬(Heap Sort)
def heapsort(iterable):
    h = []
    result = []
    # 모든 원소를 차례대로 힙에 삽입
    for value in iterable:
        heapq.heappush(h, -value)
    # 힙에 삽입된 모든 원소를 차례대로 꺼내어 담기
    for i in range(len(h)):
        result.append(-heapq.heappop(h))
    return result

result = heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0])
print(result)
```

실행 결과

```
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

18. 더 맵게 (Programmers, 우선순위큐, 최소힙)

<https://school.programmers.co.kr/learn/courses/30/lessons/42626>

- 매운 것을 좋아하는 Leo는 모든 음식의 스코빌 지수를 k 이상으로 만들고 싶습니다. 모든 음식의 스코빌 지수를 k 이상으로 만들기 위해 Leo는 스코빌 지수가 가장 낮은 두 개의 음식을 아래와 같이 특별한 방법으로 섞어 새로운 음식을 만듭니다.
- 섞은 음식의 스코빌 지수 = 가장 맵지 않은 음식의 스코빌 지수 + (두 번째로 맵지 않은 음식의 스코빌 지수 * 2)**
- Leo는 모든 음식의 스코빌 지수가 k 이상이 될 때까지 반복하여 섞습니다.
- Leo가 가진 음식의 스코빌 지수를 담은 배열 `scoville`과 원하는 스코빌 지수 k 가 주어질 때, **모든 음식의 스코빌 지수를 k 이상으로 만들기 위해 섞어야 하는 최소 횟수를 return 하도록 solution 함수를 작성해주세요.**

18. 더 맵게 (Programmers, 우선순위큐, 최소힙) 문제풀이 아이디어

scoville	K	return
[1, 2, 3, 9, 10, 12]	7	2

- 스코빌 지수가 1인 음식과 2인 음식을 섞으면 음식의 스코빌 지수가 아래와 같이 됩니다.
 - 새로운 음식의 스코빌 지수 = $1 + (2 * 2) = 5$
 - 가진 음식의 스코빌 지수 = [5, 3, 9, 10, 12]
- 스코빌 지수가 3인 음식과 5인 음식을 섞으면 음식의 스코빌 지수가 아래와 같이 됩니다.
 - 새로운 음식의 스코빌 지수 = $3 + (5 * 2) = 13$
 - 가진 음식의 스코빌 지수 = [13, 9, 10, 12]
- 모든 음식의 스코빌 지수가 7 이상이 되었고 이때 섞은 횟수는 2회입니다.

19. 보석도둑 (백준1202, 우선순위큐, 정렬, 그리디)

<https://www.acmicpc.net/problem/1202>

- 보석이 총 N 개 있다. 각 보석은 무게 M_i 와 가격 V_i 를 가지고 있다. 상덕이는 가방을 K 개 가지고 있고, 각 가방에 담을 수 있는 최대 무게는 C_i 이다. 가방에는 최대 한 개의 보석만 넣을 수 있다.
- 상덕이가 훔칠 수 있는 보석의 최대 가격을 구하는 프로그램을 작성하시오.

19. 보석도둑 (백준1202, 우선순위큐, 정렬, 그리디) 문제해결 아이디어

- N개 보석 입력 `jewels = [(무게1, 가격1), ...]`, 무게 기준 오름차순 정렬
- K개 가방 입력 `bags = [무게1, 무게2, ...]`, 무게 기준 오름차순 정렬
- `max_heap = []` # 최대 가방 무게 만큼 담길 보석 리스트
- # 각 가방에 대해서
 - 최대 가방 무게 까지 보석을 담는다. 최대 힙을 구현하도록 보석 가격의 음수를 삽입
 - 최대 가방 무게 크기 만큼 담긴 보석들 중에서 가장 비싼 보석 1개 선택해서 담는다.

20. 디스크컨트롤러(Programmers, 우선순위큐, 최소힙)

<https://school.programmers.co.kr/learn/courses/30/lessons/42627>

- 어떤 작업 요청이 들어왔을 때 **작업의 번호, 작업의 요청 시각, 작업의 소요 시간을 저장해 두는 대기 큐**가 있습니다. 처음에 이 큐는 비어있습니다.
- 디스크 컨트롤러는 **하드디스크가 작업을 하고 있지 않고 대기 큐가 비어있지 않다면 가장 우선순위가 높은 작업을 대기 큐에서 꺼내서 하드디스크에 그 작업을 시킵니다.** 이때, **작업의 소요시간이 짧은 것, 작업의 요청 시각이 빠른 것, 작업의 번호가 작은 것** 순으로 우선순위가 높습니다.
- 하드디스크는 작업을 한 번 시작하면 **작업을 마칠 때까지 그 작업만 수행합니다.**
- 하드디스크가 어떤 작업을 마치는 시점과 다른 작업 요청이 들어오는 시점이 겹친다면 **하드디스크가 작업을 마치자마자 디스크 컨트롤러는 요청이 들어온 작업을 대기 큐에 저장한 뒤 우선순위가 높은 작업을 대기 큐에서 꺼내서 하드디스크에 그 작업을 시킵니다.**
- 또, **하드디스크가 어떤 작업을 마치는 시점에 다른 작업이 들어오지 않더라도 그 작업을 마치자마자 또 다른 작업을 시작할 수 있습니다.** 이 과정에서 걸리는 시간은 없다고 가정합니다.

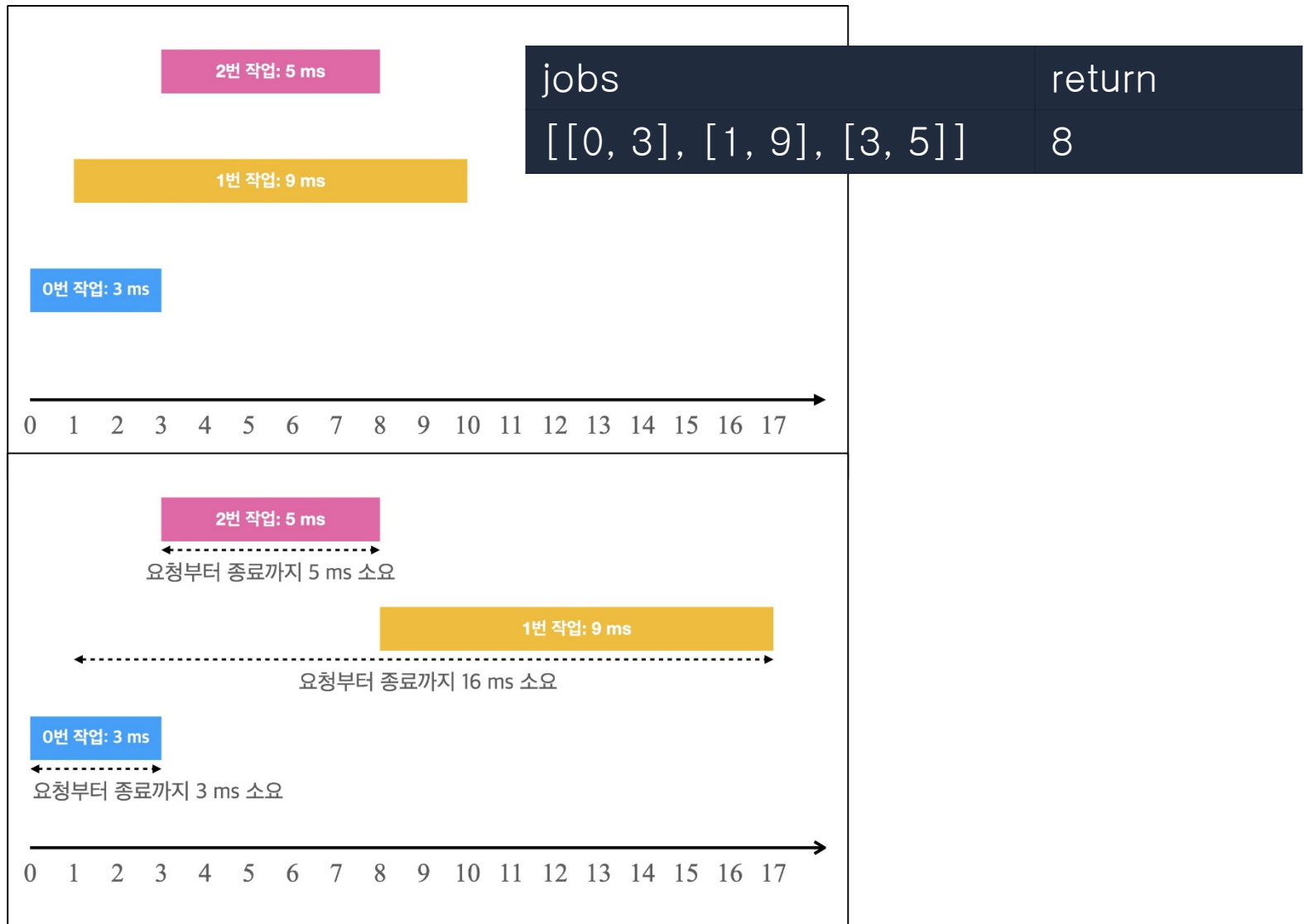
20. 디스크컨트롤러(Programmers, 우선순위큐, 최소힙)

<https://school.programmers.co.kr/learn/courses/30/lessons/42627>

- 우선순위 디스크 컨트롤러에서 **모든 요청 작업의 반환 시간의 평균은 $8\text{ms} = (3\text{ms} + 16\text{ms} + 5\text{ms}) / 3$** 가 됩니다.
- 각 작업에 대해 [작업이 요청되는 시점, 작업의 소요시간]을 담은 2차원 정수 배열 jobs가 매개변수로 주어질 때, 우선순위 디스크 컨트롤러가 이 작업을 처리했을 때 **모든 요청 작업의 반환 시간의 평균의 정수부분을 return** 하는 solution 함수를 작성해 주세요.

jobs	return
[[0, 3], [1, 9], [3, 5]]	8

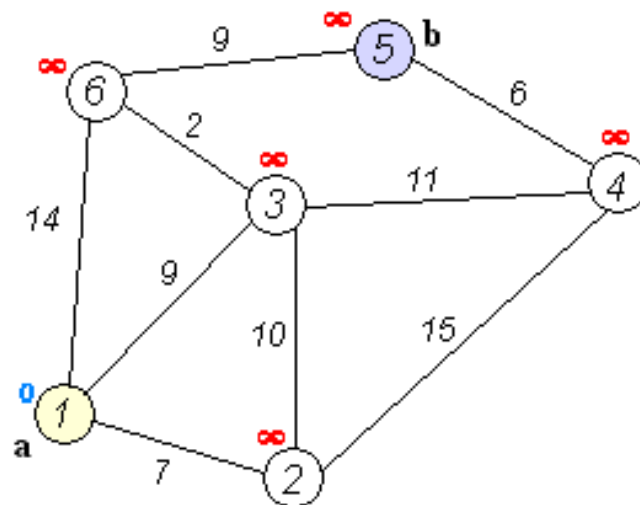
20. 디스크컨트롤러(Programmers, 우선순위큐, 최소힙) 문제풀이 아이디어



길찾기 (Dijkstra)

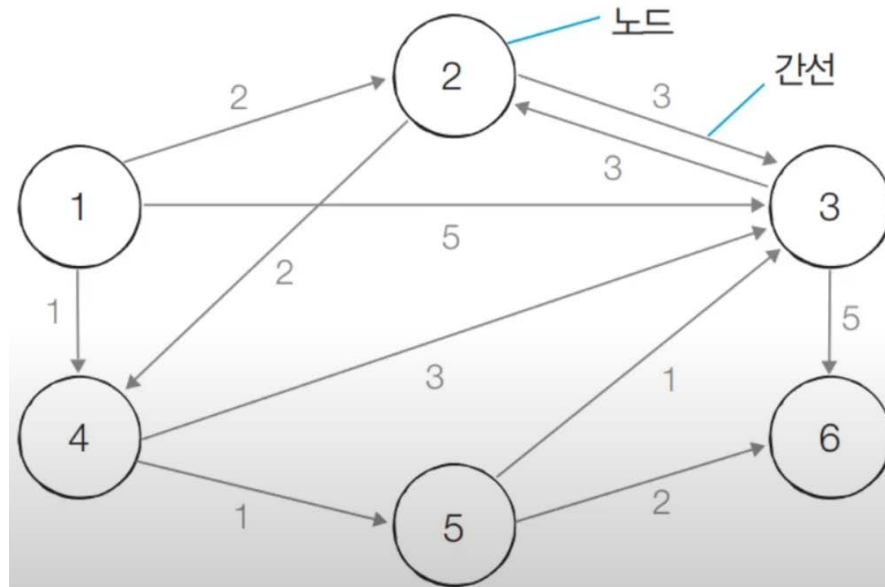
- 위키백과 다익스트라 : 시작정점에서 다른 모든정점 거리

1. 모든 꼭짓점을 미방문 상태로 표시한다. 미방문 집합이라는 모든 미방문 꼭짓점의 집합을 만든다.
2. 모든 꼭짓점에 시험적 거리 값을 부여한다: 초기점을 0으로, 다른 모든 꼭짓점을 무한대로 설정한다. 초기점을 현재 위치로 설정한다.
3. 현재 꼭짓점에서 미방문 인접 꼭짓점을 찾아 그 시험적 거리를 현재 꼭짓점에서 계산한다. 새로 계산한 시험적 거리를 현재 부여된 값과 비교해서 더 작은 값을 넣는다. 예를 들어, 현재 꼭짓점 A의 거리가 6이라고 표시되었고, 인접 꼭짓점 B로 연결되는 변의 길이가 2라고 한다면, A를 통한 B까지의 거리는 $6 + 2 = 8$ 이 된다. 이전의 B까지의 거리가 8보다 컸다면 8로 바꾸고, 그렇지 않다면 그대로 놔둔다.
4. 만약 현재 꼭짓점에 인접한 모든 미방문 꼭짓점까지의 거리를 계산했다면, 현재 꼭짓점을 방문한 것으로 표시하고 미방문 집합에서 제거한다. 방문한 꼭짓점은 이후에는 다시 방문하지 않는다.



최단 경로 문제

- 최단 경로 알고리즘은 **가장 짧은 경로를 찾는 알고리즘**을 의미
- **다양한 문제 상황**
 - 한 지점에서 다른 한 지점까지의 최단 경로
 - 한 지점에서 다른 모든 지점까지의 최단 경로
 - 모든 지점에서 다른 모든 지점까지의 최단 경로
- 각 지점은 그래프에서 **노드**로 표현
- 지점 간 연결 도로는 그래프에서 **간선**으로 표현



다익스트라 최단 경로 알고리즘 개요

- **특정한 노드에서 출발하여 다른 모든 노드로 가는 최단 경로를 계산**
- 다익스트라 최단 경로 알고리즘은 음의 간선이 없을 때 정상 동작
 - 현실 세계의 도로(간선)은 음의 간선이 표현되지 않음
- 다익스트라 최단 경로 알고리즘은 그리디 알고리즘으로 분류
 - 매 상황에서 가장 비용이 적은 노드를 선택해 임의의 과정을 반복

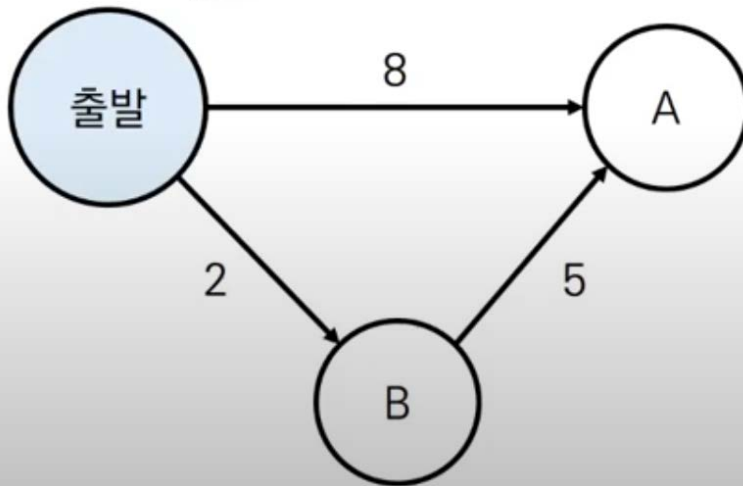
다익스트라 최단 경로 알고리즘

- 알고리즘의 **동작 과정**은 다음과 같다.
 1. 출발 노드를 설정
 2. 최단 거리 테이블을 초기화
 3. 방문하지 않은 노드 중에서 최단 거리가 가장 짧은 노드를 선택
 4. 해당 노드를 거쳐 다른 노드를 가는 비용을 계산하여 최단 거리 테이블 갱신
 5. 위의 과정에서 3번과 4번을 반복

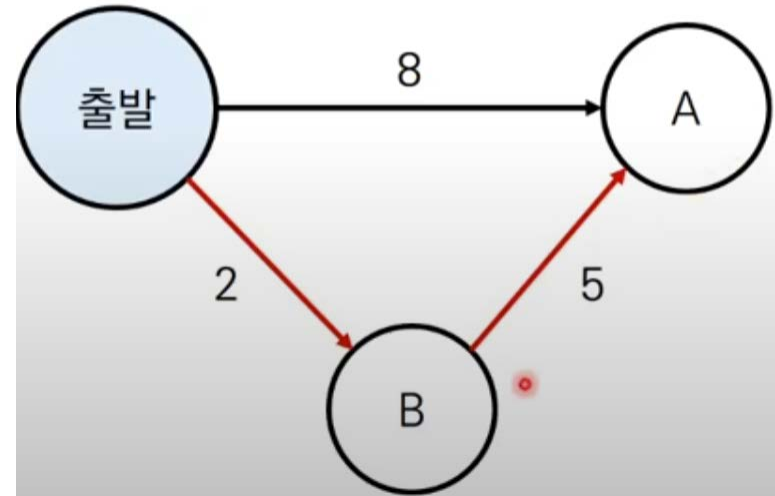
다익스트라 최단 경로 알고리즘

- 알고리즘 동작 과정에서 최단 거리 테이블은 각 노드에 대한 현재까지의 최단 거리 정보를 가지고 있다.
- 처리 과정에서 더 짧은 경로를 찾으면 ' 이제부터는 이 경로가 제일 짧은 경로야 ' 라고 갱신

A로 가는 최단 거리가 8인 것 같네.



B를 거쳐 가면 A까지 7에 갈 수 있네!



다익스트라 알고리즘: 개선된 구현 방법

- 단계마다 방문하지 않은 노드 중에서 최단 거리가 가장 짧은 노드를 선택하기 위해 힙(Heap) 자료구조를 이용
- 다익스트라 알고리즘이 동작하는 **기본원리는 동일**
 - 현재 가장 가까운 노드를 저장해 놓기 위해서 힙 자료구조를 추가적으로 이용
 - 현재의 최단 거리가 가장 짧은 노드를 선택해야 하므로 최소 힙 사용

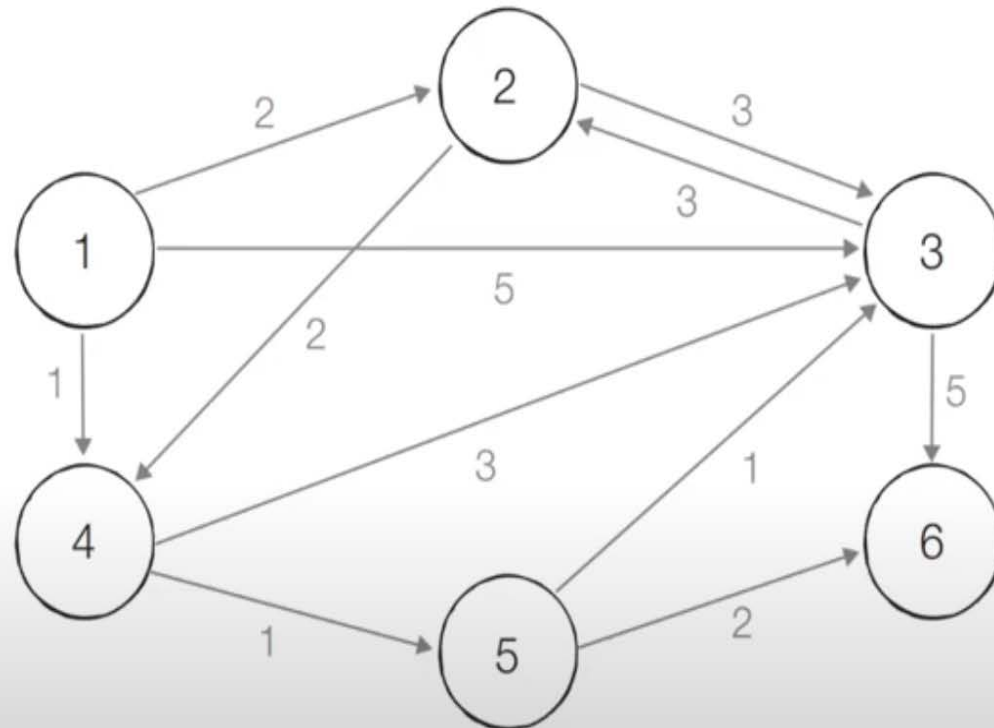
다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

[초기 상태] 그래프를 준비하고 출발 노드를 설정하여 우선순위 큐에 삽입합니다.

출발: 1번 노드

■ : 처리 중인 노드

■ : 이미 방문한 노드



우선순위 큐
(거리: 0, 노드: 1)

노드 번호	1	2	3	4	5	6
거리	0	무한	무한	무한	무한	무한

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

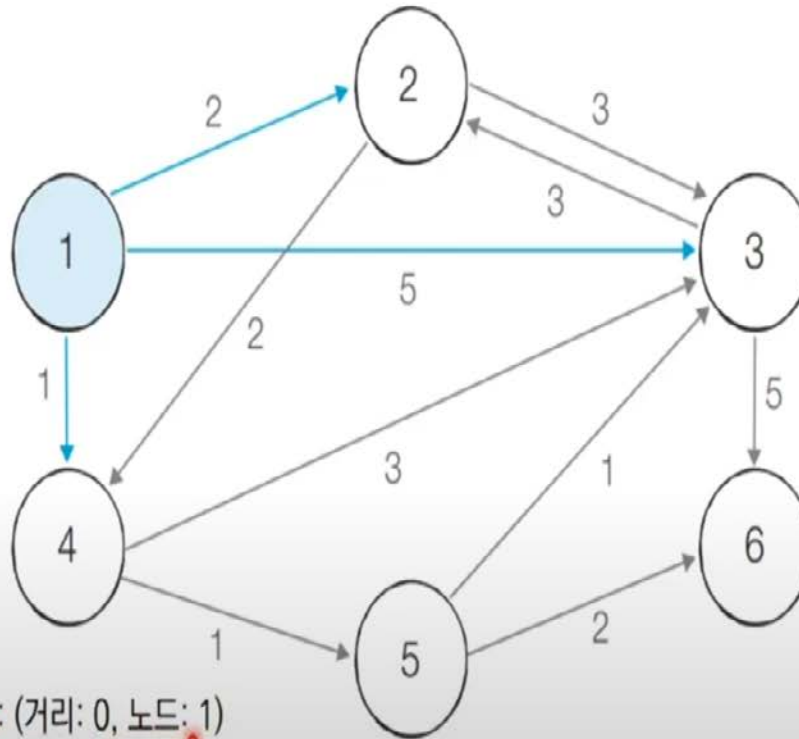
- [Step 1] 우선순위 큐에서 원소를 꺼냅니다. 1번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

■ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 1, 노드: 4)
(거리: 2, 노드: 2)
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
2번	무한	0+2	True
3번	무한	0+5	True
4번	무한	0+1	True

현재 꺼낸 원소: (거리: 0, 노드: 1)

노드 번호	1	2	3	4	5	6
거리	0	2	5	1	무한	무한

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

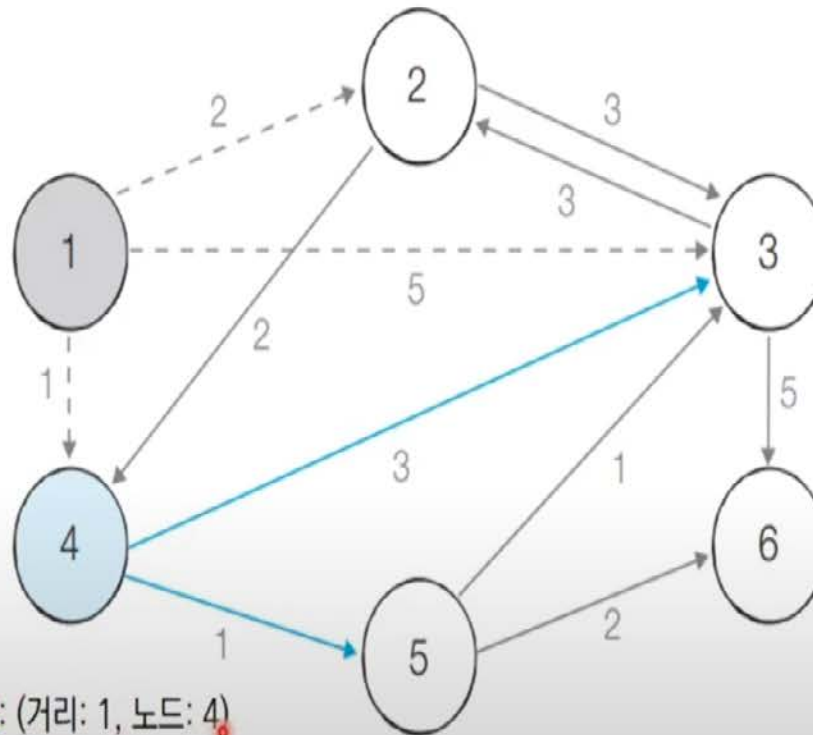
- [Step 2] 우선순위 큐에서 원소를 꺼냅니다. 4번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

■ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 2, 노드: 2)
(거리: 2, 노드: 5)
(거리: 4, 노드: 3)
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
3번	5	1+3	True
5번	무한	1+1	True

현재 꺼낸 원소: (거리: 1, 노드: 4)

노드 번호	1	2	3	4	5	6
거리	0	2	4	1	2	무한

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

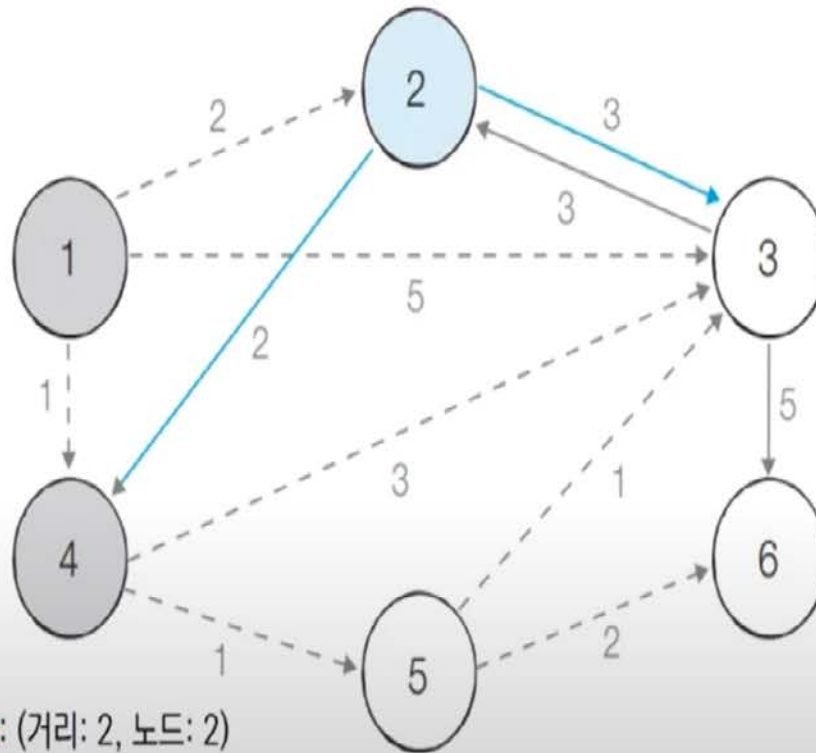
- [Step 3] 우선순위 큐에서 원소를 꺼냅니다. 2번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 2, 노드: 5)
(거리: 4, 노드: 3)
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
3번	4	2+3	False
4번	1	2+2	False

현재 꺼낸 원소: (거리: 2, 노드: 2)

노드 번호	1	2	3	4	5	6
거리	0	2	4	1	2	무한

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

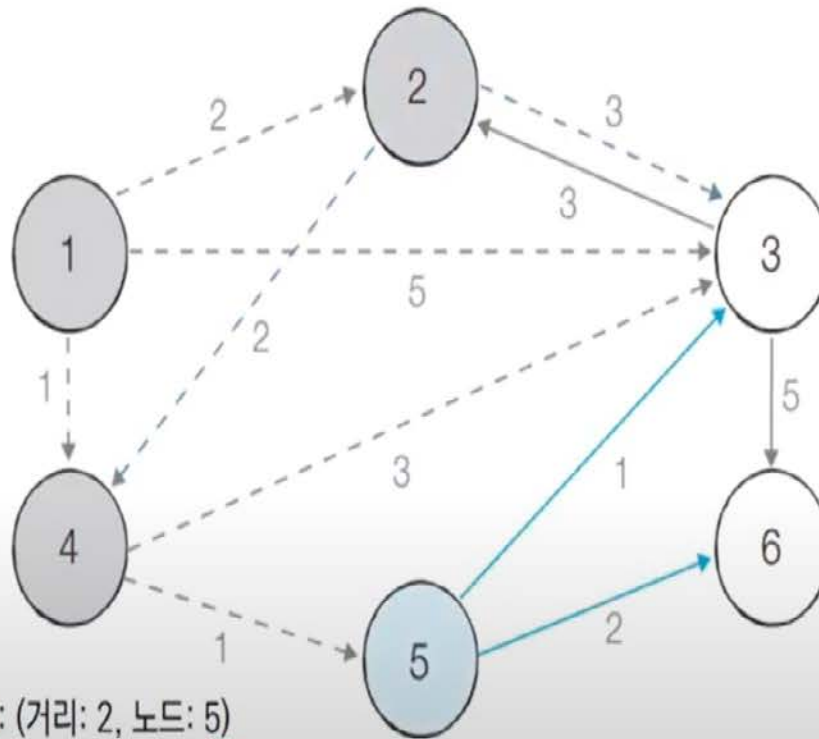
- [Step 4] 우선순위 큐에서 원소를 꺼냅니다. 5번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 3, 노드: 3)
(거리: 4, 노드: 3)
(거리: 4, 노드: 6)
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
3번	4	2+1	True
6번	무한	2+2	True

현재 꺼낸 원소: (거리: 2, 노드: 5)

노드 번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

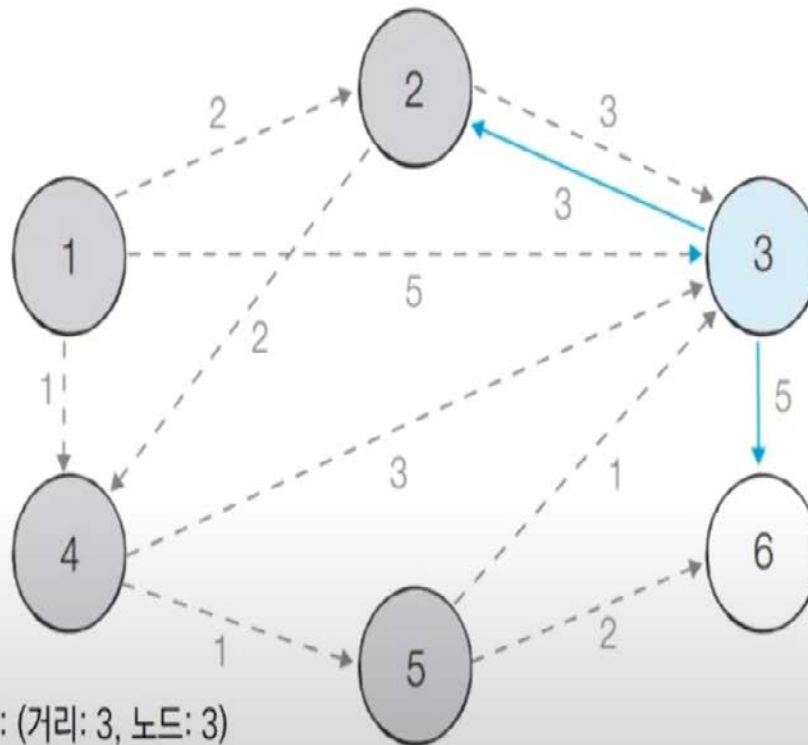
- [Step 5] 우선순위 큐에서 원소를 꺼냅니다. 3번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 4, 노드: 3)
(거리: 4, 노드: 6)
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
2번	2	3+3	False
6번	4	3+5	False

현재 꺼낸 원소: (거리: 3, 노드: 3)

노드 번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

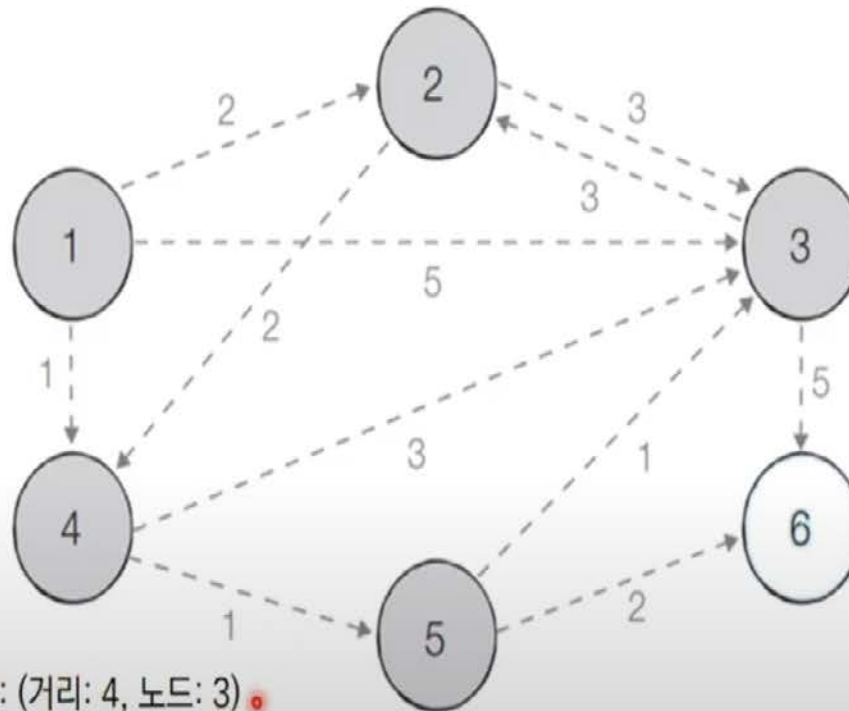
- [Step 6] 우선순위 큐에서 원소를 꺼냅니다. 3번 노드는 이미 방문했으므로 무시합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 4, 노드: 6)
(거리: 5, 노드: 3)



현재 꺼낸 원소: (거리: 4, 노드: 3) ●

노드 번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

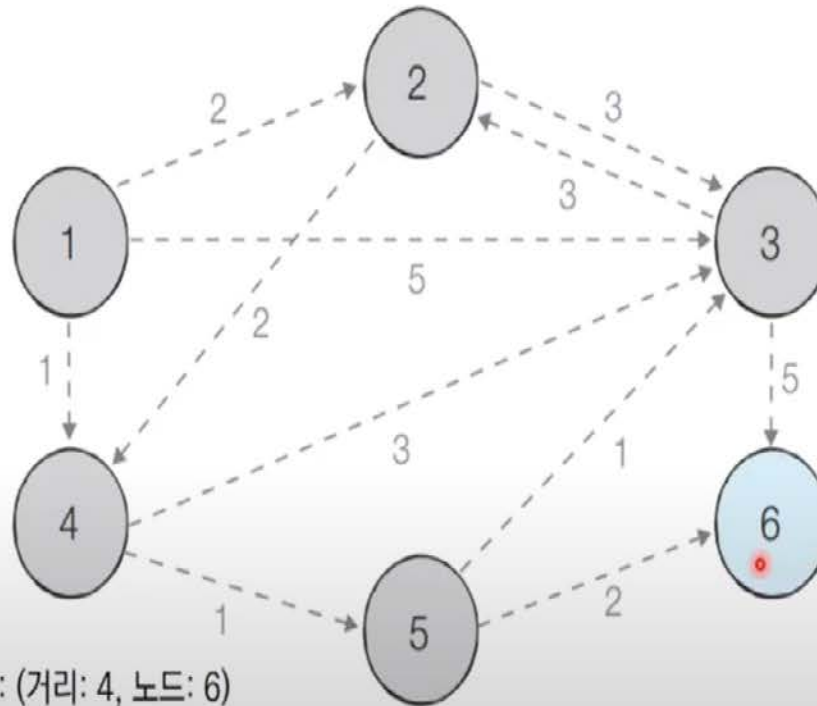
- [Step 7] 우선순위 큐에서 원소를 꺼냅니다. 6번 노드는 아직 방문하지 않았으므로 이를 처리합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐
(거리: 5, 노드: 3)



인접 노드	현재 값	거쳐갈 때	갱신 여부
이동할 수 있는 인접 노드가 없습니다.			

현재 꺼낸 원소: (거리: 4, 노드: 6)

노드 번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

다익스트라 알고리즘: 동작 과정 살펴보기 (우선순위 큐)

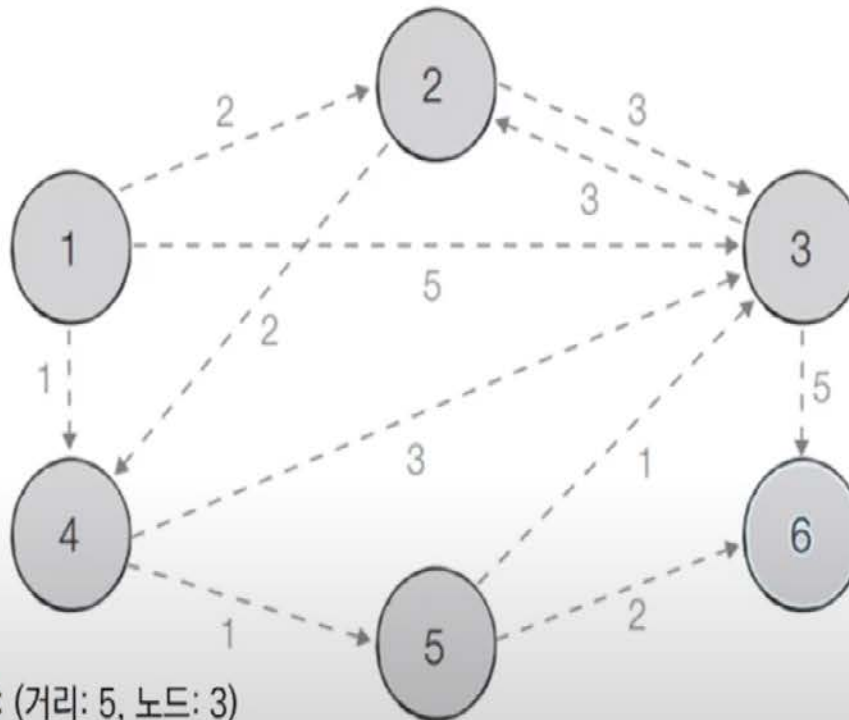
- [Step 8] 우선순위 큐에서 원소를 꺼냅니다. 3번 노드는 이미 방문했으므로 무시합니다.

출발: 1번 노드

□ : 처리 중인 노드

■ : 이미 방문한 노드

우선순위 큐



현재 꺼낸 원소: (거리: 5, 노드: 3)

노드 번호	1	2	3	4	5	6
거리	0	2	3	1	2	4

다익스트라 알고리즘: 개선된 구현 방법 (Python)

```
import heapq
import sys
input = sys.stdin.readline
INF = int(1e9) # 무한을 의미하는 값으로 10억을 설정

# 노드의 개수, 간선의 개수를 입력받기
n, m = map(int, input().split())
# 시작 노드 번호를 입력받기
start = int(input())
# 각 노드에 연결되어 있는 노드에 대한 정보를 담는 리스트를 만들기
graph = [[] for i in range(n + 1)]
# 최단 거리 테이블을 모두 무한으로 초기화
distance = [INF] * (n + 1)

# 모든 간선 정보를 입력받기
for _ in range(m):
    a, b, c = map(int, input().split())
    # a번 노드에서 b번 노드로 가는 비용이 c라는 의미
    graph[a].append((b, c))
```

```
def dijkstra(start):
    q = []
    # 시작 노드로 가기 위한 최단 경로는 0으로 설정하여, 큐에 삽입
    heapq.heappush(q, (0, start))
    distance[start] = 0
    while q: # 큐가 비어있지 않다면
        # 가장 최단 거리가 짧은 노드에 대한 정보 꺼내기
        dist, now = heapq.heappop(q)
        # 현재 노드가 이미 처리된 적이 있는 노드라면 무시
        if distance[now] < dist:
            continue
        # 현재 노드와 연결된 다른 인접한 노드들을 확인
        for i in graph[now]:
            cost = dist + i[1]
            # 현재 노드를 거쳐서, 다른 노드로 이동하는 거리가 더 짧은 경우
            if cost < distance[i[0]]:
                distance[i[0]] = cost
                heapq.heappush(q, (cost, i[0]))

# 다익스트라 알고리즘을 수행
dijkstra(start)

# 모든 노드로 가기 위한 최단 거리를 출력
for i in range(1, n + 1):
    # 도달할 수 없는 경우, 무한(INFINITY)이라고 출력
    if distance[i] == INF:
        print("INFINITY")
    # 도달할 수 있는 경우 거리를 출력
    else:
        print(distance[i])
```


다익스트라 알고리즘: 개선된 구현 방법 (C++)

```
void dijkstra(int start) {  
    priority_queue<pair<int, int> > pq;  
    // 시작 노드로 가기 위한 최단 경로는 0으로 설정하여, 큐에 삽입  
    pq.push({0, start});  
    d[start] = 0;  
    while (!pq.empty()) { // 큐가 비어있지 않다면  
        // 가장 최단 거리가 짧은 노드에 대한 정보 꺼내기  
        int dist = -pq.top().first; // 현재 노드까지의 비용  
        int now = pq.top().second; // 현재 노드  
        pq.pop();  
        // 현재 노드가 이미 처리된 적이 있는 노드라면 무시  
        if (d[now] < dist) continue;  
        // 현재 노드와 연결된 다른 인접한 노드들을 확인  
        for (int i = 0; i < graph[now].size(); i++) {  
            int cost = dist + graph[now][i].second;  
            // 현재 노드를 거쳐서, 다른 노드로 이동하는 거리가 더 짧은 경우  
            if (cost < d[graph[now][i].first]) {  
                d[graph[now][i].first] = cost;  
                pq.push(make_pair(-cost, graph[now][i].first));  
            }  
        }  
    }  
}
```

21. 지름길 (백준1446, Dijkstra)

<https://www.acmicpc.net/problem/1446>

- 매일 아침, 세준이는 학교에 가기 위해서 차를 타고 D킬로미터 길이의 고속도로를 지난다. 이 고속도로는 심각하게 커브가 많아서 정말 운전하기도 힘들다. 어느 날, 세준이는 이 고속도로에 지름길이 존재한다는 것을 알게 되었다. 모든 지름길은 일방통행이고, 고속도로를 역주행할 수는 없다.
- 세준이가 운전해야 하는 거리의 최솟값을 출력하시오.
- 첫째 줄에 지름길의 개수 N과 고속도로의 길이 D가 주어진다. N은 12 이하인 양의 정수이고, D는 10,000보다 작거나 같은 자연수이다. 다음 N개의 줄에 지름길의 시작 위치, 도착 위치, 지름길의 길이가 주어진다.

예제 입력 1 복사

```
5 150
0 50 10
0 50 20
50 100 10
100 151 10
110 140 90
```

예제 출력 1 복사

```
70
```

21. 지름길 (백준1446, Dijkstra)

문제해결 아이디어

- $D(\text{목적지}) + 1$ 개 $(0, 1, 2, \dots, D)$ 노드를 생성하고 인접 노드 끼리 거리를 1로 설정
- N 개 지름길 (a, b) 에지와 $weight(\text{거리})$ 를 설정
- 시작점(0) -> 목적지(D) 최단거리 Dijkstra 알고리즘

22. 숨바꼭질3 (백준13549, Dijkstra)

<https://www.acmicpc.net/problem/13549>

- 수빈이는 동생과 숨바꼭질을 하고 있다. 수빈이는 현재 점 $N(0 \leq N \leq 100,000)$ 에 있고, 동생은 점 $K(0 \leq K \leq 100,000)$ 에 있다.
- 수빈이는 걷거나 순간이동을 할 수 있다. 만약, 수빈이의 위치가 X 일 때 걷는다면 1초 후에 $X-1$ 또는 $X+1$ 로 이동하게 된다. 순간이동을 하는 경우에는 0초 후에 $2 \times X$ 의 위치로 이동하게 된다.
- 수빈이와 동생의 위치가 주어졌을 때, 수빈이가 동생을 찾을 수 있는 가장 빠른 시간이 몇 초 후인지 구하는 프로그램을 작성하시오.
- 첫 번째 줄에 수빈이가 있는 위치 N 과 동생이 있는 위치 K 가 주어진다. N 과 K 는 정수이다.

23. K번째 최단경로 찾기 (백준1854, Dijkstra)

<https://www.acmicpc.net/problem/1854>

- 첫째 줄에 n, m, k 가 주어진다. ($1 \leq n \leq 1,000, 0 \leq m \leq 250,000, 1 \leq k \leq 100, mk \leq 3,000,000$) n 과 m 은 각각 김 조교가 여행을 고려하고 있는 도시들의 개수와, 도시 간에 존재하는 도로의 수이다.
- 이어지는 m 개의 줄에는 각각 도로의 정보를 제공하는 세 개의 정수 a, b, c 가 포함되어 있다. 이것은 a 번 도시에서 b 번 도시로 갈 때는 c 의 시간이 걸린다는 의미이다. ($1 \leq a, b \leq n, 1 \leq c \leq 1,000$)
- 도시의 번호는 1번부터 n 번까지 연속하여 붙어 있으며, 1번 도시는 시작도시이다. 두 도로의 시작점과 도착점이 모두 같은 경우는 없다.
- n 개의 줄을 출력한다. i 번째 줄에 1번 도시에서 i 번 도시로 가는 k 번째 최단경로의 소요시간을 출력한다.
- k 번째 최단경로가 존재하지 않으면 -1을 출력한다.

23. K번째 최단경로 찾기 (백준1854, Dijkstra)

문제해결 아이디어

- 각 노드의 최단 거리 테이블 (distance)이 K개 최단 거리를 갖도록 2차원 배열로 설정
- 새로운 최단 거리가 최단 거리 테이블 의 K번째 최단 거리보다 작을 때 최단 거리 테이블을 갱신

24. 장애물 (TUKorea0073, Dijkstra)

<http://210.93.60.51/problem/0073>

- 정점의 개수 V 와 간선의 개수 E : 양방향 그래프
- 원래 최단 경로의 길이 A 를 구한다.
- 그런 후 임의의 간선에 장애물이 떨어져서 가중치가 2배가 되었다고 가정하고 최단 경로 B 를 구한다.
- 최종적으로 $B-A$ 의 최대값을 구한다.

24. 장애물 (TUKorea0073, Dijkstra)

<http://210.93.60.51/problem/0073>

- # 1. 다익스트라 알고리즘으로 오리지널 최단경로를 구한다.
- # 2. 다익스트라 최단 경로의 path를 알아낸다. 각 정점의 이전 정점을 기록하는 previous를 사용한다.
- # 목적지 V-1->previous[V-1]->previous[previous[V-1]] -> 0이 나오기까지 따라 간다.
- # 3. 최단 경로 path의 에지들을 doubling 한 최단경로와 original 최단 경로 차이의 최대값을 구한다.
- # E개 간선을 모두 2배로 하면서 최대값을 구하면 시간초과 걸린다.
- # 최단 경로의 간선만 2배로 해본다.

24. 장애물 (TUKorea0073, Dijkstra)

<http://210.93.60.51/problem/0073>

```
# Adjacency Graph
# 2차원 딕셔너리로 하면 2차원 리스트와 달리 각 정점에서 이웃 정점을 바로 검색할 수 있는 장점이 있다.
graph = {} #2차원 딕셔너리, key:value 정점:{이웃정점1:weight1, 이웃정점2:weight2, ... }

for i in range(V): #i=0,1,2,...V-1
    graph[i] = {}
for _ in range(E): #i=0,1,2,...E-1 에지개수 만큼
    u,v,w = map(eval, input().split())
    graph[u-1][v-1] = w
    graph[v-1][u-1] = w
#print(graph)
# {0: {1: 5, 2: 1}, 1: {0: 5, 2: 8, 3: 7}, 2: {0: 1, 1: 8, 4: 7, 3: 3}, 3: {2: 3, 1: 7, 4: 2},
# 4: {2: 7, 3: 2}}
```

길찾기 (Floyd Warshall)

- 위키백과 Floyd Warshall
- 파이썬 구현 <https://it-garden.tistory.com/247>
- 모든 (i,j) 쌍 간의 최단 경로를 구함

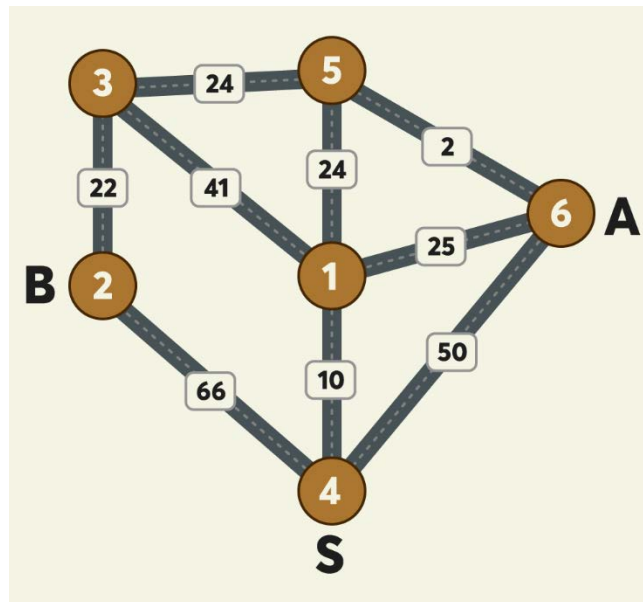
1. 하나의 정점에서 다른 정점으로 바로 갈 수 있으면 최소 비용을, 갈 수 없다면 INF로 배열에 값을 저장합니다.
2. 3중 for문을 통해 거쳐가는 정점을 설정한 후 해당 정점을 거쳐가서 비용이 줄어드는 경우에는 값을 바꿔줍니다.
3. 위의 과정을 반복해 모든 정점 사이의 최단 경로를 탐색합니다.

```
def Floyd_Warshall():  
    dist = [[INF]*n for i in range(n)] # 최단 경로를 담는 배열  
  
    for i in range(n): # 최단 경로를 담는 배열 초기화  
        for j in range(n):  
            dist[i][j] = a[i][j]  
  
    for k in range(n): # 거치는 점  
        for i in range(n): # 시작점  
            for j in range(n): # 끝점  
                # k를 거쳤을 때의 경로가 더 적은 경로  
                if dist[i][j] > dist[i][k] + dist[k][j]:  
                    dist[i][j] = dist[i][k] + dist[k][j]
```

25. 합승 택시 요금 (Programmers, Floyd Warshall)

<https://school.programmers.co.kr/learn/courses/30/lessons/72413>

- 2021 카카오공채 1차 4번
- <https://tech.kakao.com/2021/01/25/2021-kakao-recruitment-round-1/>
- 다익스트라 : K정점에서 모든 정점까지 최단거리 구함
- Floyd Warshall : (i,j) 모든 정점 쌍의 최단거리 구함



25. 합승 택시 요금 (Programmers, Floyd Warshall)

<https://school.programmers.co.kr/learn/courses/30/lessons/72413>

```
# STEP2-1 다익스트라를 이용한 모든 (i,j)쌍 최단거리 구함
d = [[INF] * n for _ in range(n)] # nxn 노드에 대해서 최단거리를 갖는 2차원 배열 INF로 초기화
for i in range(n):
    d[i] = dijkstra(i, graph) # 정점i에서 시작해서 다른 모든 정점까지의 최단 거리

# STEP2-2 Floy Warshall를 이용한 모든 (i,j)쌍 최단거리 구함
# 유형 그래프 필요없음
d = Floyd_Warshall(n, fares)
```

```
#STEP3 min(d[s][k] + d[k][a] + d[k][b]) 의 최소값 구함
# s노드에서 k노드까지 합승하고 나서 k에서 a와 b로 각자 간다고 할 때
```

26. 두수의 합 (백준3273 정렬, 투포인터)

<https://www.acmicpc.net/problem/3273>

- n 개의 서로 다른 양의 정수 $a_1, a_2, \dots, a_n$ 으로 이루어진 수열이 있다. a_i 의 값은 1보다 크거나 같고, 1000000보다 작거나 같은 자연수이다. 자연수 x 가 주어졌을 때, $a_i + a_j = x$ ($1 \leq i < j \leq n$)을 만족하는 (a_i, a_j) 쌍의 수를 구하는 프로그램을 작성하시오.
- 첫째 줄에 수열의 크기 n 이 주어진다. 다음 줄에는 수열에 포함되는 수가 주어진다. 셋째 줄에는 x 가 주어진다. ($1 \leq n \leq 100000, 1 \leq x \leq 2000000$)
- 문제의 조건을 만족하는 쌍의 개수를 출력한다.

예제 입력 1 복사

```
9
5 12 7 10 9 1 2 3 11
13
```

예제 출력 1 복사

```
3
```

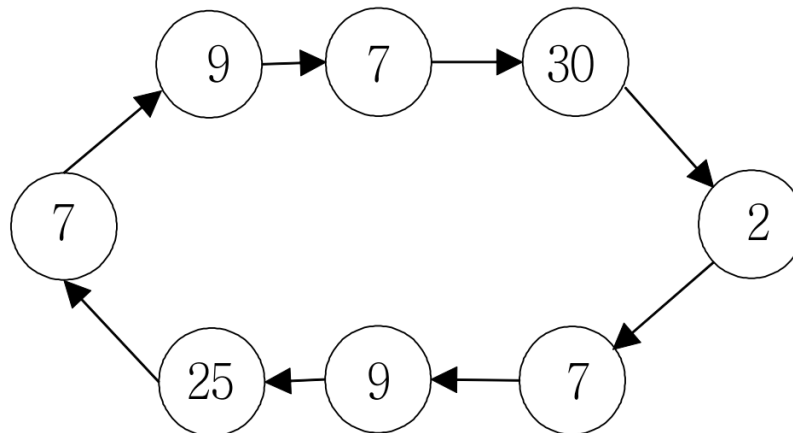
26. 두수의 합 (백준3273, 정렬, 투포인터) 문제해결 아이디어

- 오름 차순 정렬
 - $a = [5, 12, 7, 10, 9, 1, 2, 3, 11]$
 - $a = [1, 2, 3, 5, 7, 9, 10, 11, 12]$
- **start = 0, 1, 2, ..., n-2, start를 차례로 이동시키면서**
 - $a[start] + a[end]$ 값이 x 보다 작다면 end 를 가능한 만큼 오른쪽 이동
 - $a[start] + a[end]$ 값이 x 보다 크다면 end 를 가능한 만큼 왼쪽 이동

27. 회전초밥 (백준2531, 투포인터, 슬라이딩윈도우)

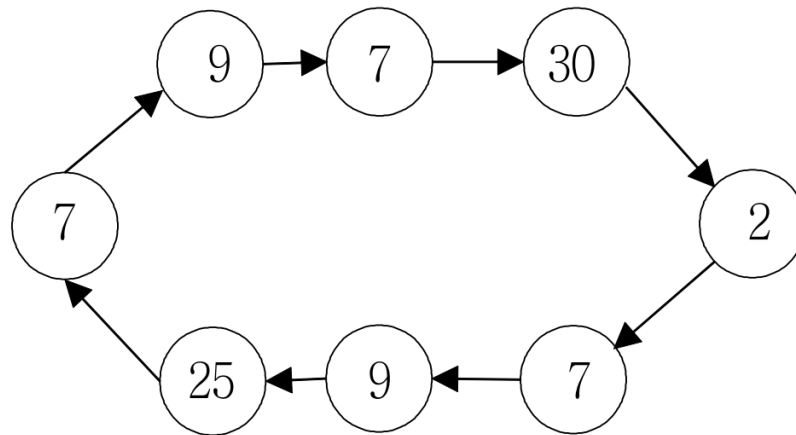
<https://www.acmicpc.net/problem/2531>

- 원래 회전 초밥은 손님이 마음대로 초밥을 고르고, 먹은 초밥만큼 식대를 계산하지만, 벨트의 임의의 한 위치부터 k 개의 접시를 연속해서 먹을 경우 할인된 정액 가격으로 제공한다.
- 각 고객에게 초밥의 종류 하나가 쓰인 쿠폰을 발행하고, 1번 행사에 참가할 경우 이 쿠폰에 적혀진 종류의 초밥 하나를 추가로 무료로 제공한다.
- 회전 초밥 음식점의 벨트 상태, 메뉴에 있는 초밥의 가짓수, 연속해서 먹는 접시의 개수, 쿠폰 번호가 주어졌을 때, 손님이 먹을 수 있는 초밥 가짓수의 최댓값을 구하는 프로그램을 작성하시오.



27. 회전초밥 (백준2531, 투포인터, 슬라이딩 윈도우) 문제해결 아이디어

- 슬라이딩 윈도우
- 투포인터
- 브루트포스 알고리즘
- **start, end 두개 포인터를 k간격을 갖도록 이동하면서 최대 초밥 가짓수를 설정**



28. 겹치는 건 싫어 (백준20922, 투포인터, 슬라이딩 윈도우)

<https://www.acmicpc.net/problem/20922>

- 100,000 이하의 양의 정수로 이루어진 길이가 N 인 수열이 주어진다. 이 수열에서 **같은 정수를 k 개 이하로 포함한 최장 연속 부분 수열의 길이**를 구하는 프로그램을 작성해보자.
- 첫째 줄에 정수 N ($1 \leq N \leq 200,000$)과 K ($1 \leq K \leq 100$)가 주어진다.
- 둘째 줄에는 $\{a_1, a_2, \dots, a_n\}$ 이 주어진다 ($1 \leq a_i \leq 100,000$)
- 조건을 만족하는 최장 연속 부분 수열의 길이를 출력한다.

누적합(Prefix Sum)

- 배열 또는 리스트 등에서 일정 구간의 합을 빠르게 계산하기 위한 방법이면서 동적 계획법(DP)의 형태 중 하나이다.
- 기본적인 방식은 각 요소까지의 누적 합을 계산하여 이를 배열에 저장해 두는 것이다. 이후에 특정 구간의 합을 구할 때는 해당 구간의 끝 지점까지의 누적 합에서 시작 지점까지의 누적 합을 빼는 것이다.
-
- 해당 알고리즘은 배열 또는 리스트의 요소가 고정되어 있을 때 구간 합을 반복적으로 계산해야 하는 경우 유용하게 사용될 수 있다.

29. 블로그 (백준21921, 슬라이딩윈도우, 누적합)

<https://www.acmicpc.net/problem/21921>

- 찬솔이는 x 일 동안 가장 많이 들어온 방문자 수와 그 기간들을 알고 싶다.
- x 일 동안 가장 많이 들어온 방문자 수를 출력한다. 만약 최대 방문자 수가 0명이라면 SAD를 출력한다.
- 만약 최대 방문자 수가 0명이 아닌 경우 둘째 줄에 기간이 몇 개 있는지 출력한다.



30. 목재총량 (TUKorea0044, 누적합, DP)

<http://210.93.60.51/problem/0044>

3	4	15	23
14	20	12	9
3	8	12	15
12	20	7	5

(2,2) 와 (3,4) 모서리를 가지는 직사각형

지정한 직사각형 토지의 목재 총량을 계산하는 것입니다.

(r1,c1)은 좌측 상단 모서리, (r2,c2)는 우측 하단 모서리를 나타냅니다.

목재 총량을 계산해야 하는 직사각형 토지 개수 C가 매우 큰 수 입니다.

30. 목재총량 (TUKorea0044, 누적합, DP)

<http://210.93.60.51/problem/0044>

```
-----  
      |               |  
-(r1-1,c1-1)------(r1-1,c2)-----  
-----|---(r1,c1)-----|-----  
      |       |       |  
      |       |       |  
---(r2,c1-1)------(r2,c2)-----
```

`R = []` #M행xN열 2차원배열 나무

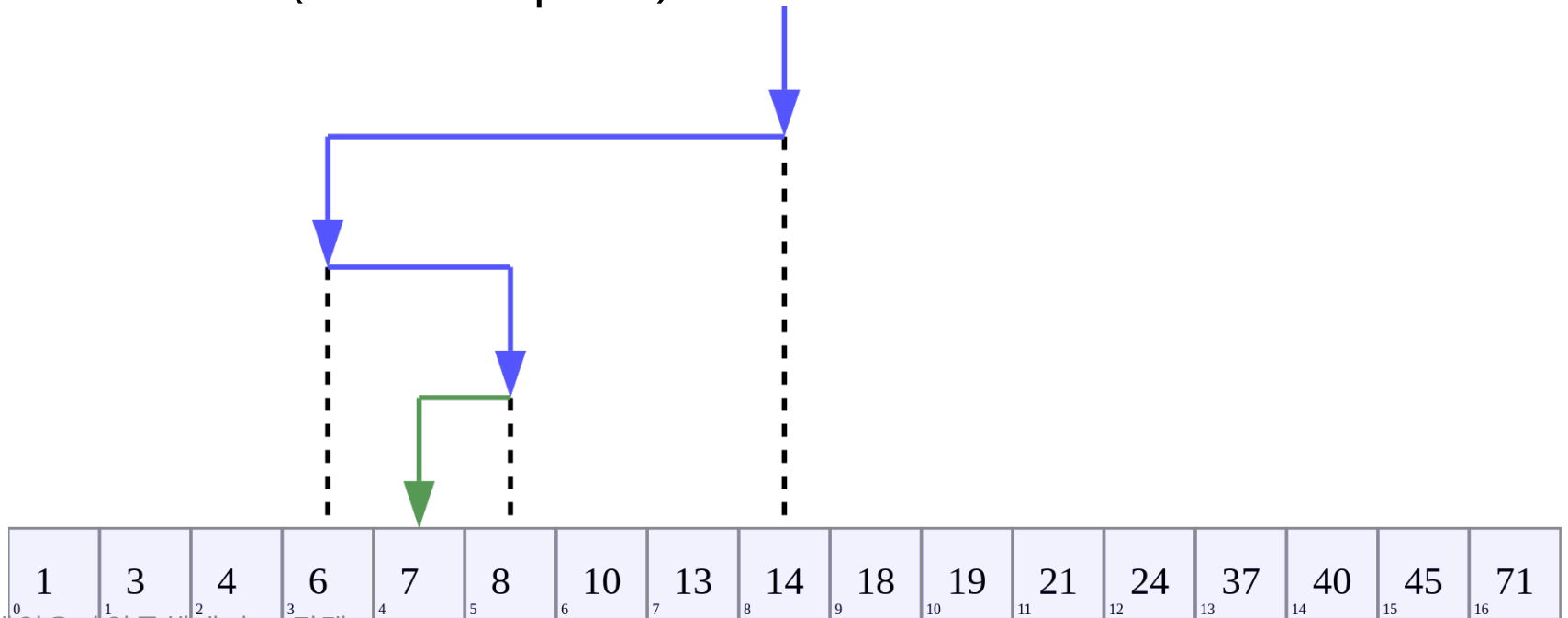
`Q[r][c]`는 $(0,0)$ 에서 (r,c) 직사각형 목재의 총량

$Q[r][c] = Q[r][c-1] + Q[r-1][c] - Q[r-1][c-1] + R[r][c]$

$[r1,c1,r2,c2]$ 목재총량 = $Q(r2,c2) - Q(r1-1,c2) - Q(r2,c1-1) + Q(r1-1,c1-1)$

이분탐색 (Binary Search)

- 위키백과 이진 검색
- 오름차순으로 정렬된 리스트에서 특정한 값의 위치를 찾는 알고리즘
- 중간값과 찾고자 하는 값의 크고 작음을 비교하여 탐색 범위(Search Space)를 $\frac{1}{2}$ 로 줄임



31. 나무 자르기 (백준2805, 이분탐색)

<https://www.acmicpc.net/problem/2805>

- M미터 이상 나무를 가져가기 위한 절단기 높이 H 최대값.
- 나무들을 정렬하지 않아도 된다.
- 절단기 높이 H는 [0:1000000000] 사이의 값으로 이미 정렬되어 있는 것임

```
start = 0
end = 1000000000 #max(array)
# 조건을 만족하는 절단기 최대높이
answer = 0
# 이분 탐색
while start <= end:
    mid = (start + end) // 2
    if check(mid):      # 조건을 만족하면
        answer = mid    # 정답후보
        start = mid + 1  # 절단기 높이를 올린다.
    else:               # 조건을 만족하지 않으면
        end = mid - 1    # 절단기 높이를 낮춘다.
print(answer)
```

32. 셔틀버스 대기시간 (TUKorea0069, 이분탐색)

<http://210.93.60.51/problem/0069>

- 당신의 임무는 셔틀버스 배차 시간을 최적으로 조정하여 도착하는 학생의 최대 대기시간 중에 가장 작은 값을 알아내는 것입니다.

```
# 문제를 변형하여  
# 최대 대기시간의 Search Space 를 [0:1000000000] 로 설정하고  
# 이분 탐색하여 mid 대기시간으로 M대 셔틀버스에 N명을 수용할 수 있는가 check  
# 하면서 최대 대기시간의 최솟값 구함
```


32. 셔틀버스 대기시간 (TUKorea0069, 이분탐색)

<http://210.93.60.51/problem/0069>

```
int start = 0;
int end = 1000000000;
int result = 0;
while (start <= end) {
    int mid = (start + end) / 2;
    // mid 최대대기시간으로 N명의 학생들을 M대 셔틀버스에 배차할 수 있는 가?
    if (check(mid)) {
        result = mid;    // 정답후보 기록
        end = mid - 1;  // 대기시간을 줄임
    }
    else
        start = mid + 1; // 대기시간을 늘임
}
cout << result << endl;
```

다음 셔틀 배차되는 조건 2가지

1. 첫번째 사람과 i번째 사람의 도착시간 차이가 wait 보다 크면 i번째 사람은 다음 셔틀에 배차됨

2. 첫번째 사람부터 i번째 사람까지 명수가 c보다 크면 i번째 사람은 다음 셔틀에 배차됨

33. 공유기 설치 (백준2110, 이분탐색)

<https://www.acmicpc.net/problem/2110>

- 집 N 개가 수직선 위에 있다. 각각의 집의 좌표는 $x_1, \dots, x_N$ 이고, 집 여러개가 같은 좌표를 가지는 일은 없다.
- 집에 공유기 C 개를 설치하려고 한다. 한 집에는 공유기를 하나만 설치할 수 있다.
- C 개의 공유기를 N 개의 집에 적당히 설치해서, 가장 인접한 두 공유기 사이의 거리를 최대로 하는 프로그램을 작성하시오.
- 첫째 줄에 집의 개수 N ($2 \leq N \leq 200,000$)과 공유기의 개수 C ($2 \leq C \leq N$)이 하나 이상의 빈 칸을 사이에 두고 주어진다.
- 둘째 줄부터 N 개의 줄에는 집의 좌표를 나타내는 x_i ($0 \leq x_i \leq 1,000,000,000$)가 한 줄에 하나씩 주어진다.